

Introducción a Javascript

Índice

1 - Tema 1 - Introducción y programación básica.....	6
1.1 ¿Qué es JavaScript?.....	6
1.2 Cómo incluir JavaScript en documentos HTML.....	6
1.2.1 En el mismo documento.....	6
1.2.2 En un archivo externo (recomendado).....	6
1.2.3 Dentro de elementos del HTML.....	7
1.3 Ubicación de las etiquetas script.....	7
1.4 La etiqueta noscript.....	8
1.5 Detectar errores en JavaScript.....	8
1.5.1 Consola y mensajes de error.....	9
1.6 Sintaxis.....	9
1.7 El primer script.....	10
Ejercicio 1.1.....	10
1.8 Programación básica.....	10
1.8.1 Comunicación con el usuario.....	11
1.8.2 Variables.....	11
1.8.3 Tipos de variables.....	12
Ejercicio 1.2.....	13
1.8.4 Booleanos.....	14
1.8.5 Conversiones entre tipos simples.....	14
Ejercicio 1.3.....	16
1.8.6 Operadores.....	16
1.9 Estructuras de control de flujo.....	20

1.9.1 Estructura if.....	20
1.9.2 Estructura if...else.....	21
1.9.3 Estructura switch.....	22
Ejercicio 1.4.....	22
1.9.4 Estructura for.....	23
1.9.5 Estructura While.....	23
1.9.6 estructura do...while.....	24
1.9.7 Sentencias break y continue.....	25
Ejercicio 1.5.....	26
Ejercicio 1.6.....	26
2 - Tema 2 - Funciones.....	27
2.1 Definición de funciones.....	27
2.1.1 Uso de parámetros.....	27
Ejercicio 2.1.....	28
2.1.2 Número de parámetros y parámetros por defecto.....	28
2.1.3 Tipo de retorno.....	29
Ejercicio 2.2.....	29
Ejercicio 2.3:.....	30
2.1.4 Variables locales y globales.....	30
2.2 Algunas funciones útiles.....	30
2.2.1 Funciones para manejo de textos.....	30
Ejercicio 2.4:.....	32
2.2.2 Funciones matemáticas.....	32
2.2.3 Trabajando con fechas.....	33
2.3 Opciones avanzadas con funciones.....	33
2.3.1 Funciones anónimas.....	33
2.3.2 Funciones lambda o arrow functions.....	34
Ejercicio 2.5:.....	35
2.3.3 Cargar diversos archivos fuente.....	35
Ejercicio 2.6.....	35
Ejercicio 2.7.....	36
Ejercicio 2.8.....	36
Ejercicio 2.9.....	36



Ejercicio 2.10.....	37
3 Tema 3 - Gestión de arrays y datos complejos.....	38
3.1 Manipulación básica de arrays en JavaScript.....	38
3.1.1 Arrays y referencias.....	39
3.2 2. Recorrido de arrays.....	40
3.3 Operaciones con arrays.....	40
Ejercicio 1:.....	41
3.4 Ordenación de arrays.....	42
Ejercicio 2:.....	42
3.4.1 Otras operaciones avanzadas con arrays.....	43
Ejercicio 3:.....	44
Ejercicio 4:.....	44
3.5 Objetos JavaScript.....	44
Ejercicio 5:.....	45
Ejercicio 6.....	45
Ejercicio 7.....	45
Ejercicio 8.....	45
Ejercicio 9.....	45
4 Tema 4 - Acceso a los elementos de un documento DOM.....	46
4.1 Introducción. BOM y DOM.....	46
4.2 Elementos principales del BOM.....	46
4.2.1 Uso de timeouts e intervalos.....	47
4.3 Uso del DOM.....	48
4.3.1 Instrucciones para explorar el DOM.....	48
Ejercicio 1:.....	50
4.3.2 Uso del query selector.....	51
Ejercicio 2:.....	51
4.3.3 Manipulación del DOM.....	52
Ejercicio 3:.....	54
Ejercicio 4:.....	55
Ejercicio 5.....	55
Ejercicio 6.....	55
5 Tema 5 - Gestión de eventos.....	56



5.1 Eventos y programación tradicional.....	56
5.2 Principales tipos de eventos.....	56
5.2.1 Eventos sobre el contenido de la página (body).....	56
5.2.2 Eventos de teclado.....	57
5.2.3 Eventos de ratón.....	57
5.2.4 Eventos de formulario.....	57
5.2.5 Otros eventos.....	58
5.3 Uso básico de eventos.....	58
5.3.1 Manejadores de eventos mediante atributos.....	58
5.3.2 Manejadores de eventos mediante atributos y funciones.....	58
5.3.3 Manejadores de eventos usando el DOM.....	59
5.3.4 Manejadores de eventos con event listeners.....	60
Ejercicio 4.1:.....	61
Ejercicio 4.2:.....	61
4. Uso avanzado de eventos.....	61
5.3.5 Propagación de eventos.....	61
Ejercicio 4.3.....	63
5.3.6 Obtener información del evento.....	63
Ejercicio 4.4:.....	64
5.3.7 El evento de “arrastrar y soltar” (drag & drop).....	65
Ejercicio 5.5.....	67
5.3.8 Encapsulando el código JavaScript.....	68
Ejercicio 5.6.....	69
Ejercicio 5.1 (antiguo material).....	69
Ejercicio 5.2 (antiguo material).....	69
Ejercicio 5.3 (antiguo material).....	70
6 Tema 6 - Validación de formularios.....	71
6.1 Validación tradicional de formularios.....	71
6.1.1 Acceso a los datos del formulario.....	71
6.1.2 Consulta y comprobación de valores.....	72
Ejercicio 1:.....	74
6.1.3 Algunas opciones de validación de campos.....	75
6.1.4 Cómo validar todo el formulario.....	77



Ejercicio 2:.....	78
2. Validación de formularios combinada con HTML5.....	79
6.1.5 Validaciones personalizadas.....	80
6.1.6 Personalizar mensajes de error.....	81
6.1.7 CSS y validación de formularios.....	82



1 - Tema 1 - Introducción y programación básica

1.1 ¿Qué es JavaScript?

JavaScript es un lenguaje de programación que se utiliza principalmente para crear páginas web dinámicas.

Una página web dinámica es aquella que incorpora efectos como texto que aparece y desaparece, animaciones, acciones que se activan al pulsar botones y ventanas con mensajes de aviso al usuario.

Técnicamente, JavaScript es un lenguaje de programación interpretado, por lo que no es necesario compilar los programas para ejecutarlos. En otras palabras, los programas escritos con JavaScript se pueden probar directamente en cualquier navegador sin necesidad de procesos intermedios.

A pesar de su nombre, JavaScript no guarda ninguna relación directa con el lenguaje de programación Java. Legalmente, JavaScript es una marca registrada de la empresa Sun Microsystems, como se puede ver en <http://www.sun.com/suntrademarks/>.

1.2 Cómo incluir JavaScript en documentos HTML

1.2.1 En el mismo documento

El código JavaScript se encierra entre etiquetas `<script>` y se incluye en cualquier parte del documento. Aunque es correcto incluir cualquier bloque de código en cualquier zona de la página, se recomienda definir el código JavaScript dentro de la cabecera del documento (dentro de la etiqueta `<head>`):

```
<!DOCTYPE html >
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Ejemplo de código JavaScript en el propio documento</title>
    <script>
      Console.log("Hola mundo");
    </script>
  </head>
  <body>
    <p>Un párrafo de texto.</p>
  </body>
</html>
```

1.2.2 En un archivo externo (recomendado)

Archivo `codigo.js` `alert("Un mensaje de prueba");`

Documento HTML

```
<!DOCTYPE html>
```



```
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Ejemplo de código JavaScript en el propio documento</title>
    <script src="/js/codigo.js"></script>
  </head>
  <body>
    <p>Un párrafo de texto.</p>
  </body>
</html>
```

Los archivos de tipo JavaScript son documentos normales de texto con la extensión .js, que se pueden crear con cualquier editor de texto como Visual Studio Code (el que nosotros utilizaremos), Notepad++, Vi, etc.

La principal ventaja de enlazar un archivo JavaScript externo es que se simplifica el código HTML de la página, que se puede reutilizar el mismo código JavaScript en todas las páginas del sitio web y que cualquier modificación realizada en el archivo JavaScript se ve reflejada inmediatamente en todas las páginas HTML que lo enlazan.

1.2.3 Dentro de elementos del HTML

Este último método es el menos utilizado, ya que consiste en incluir trozos de JavaScript dentro del código HTML de la página:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Ejemplo de código JavaScript en el propio documento</title>
  </head>
  <body>
    <p onclick="alert('Un mensaje de prueba')">Un párrafo de texto.</p>
  </body>
</html>
```

El mayor inconveniente de este método es que *ensucia* innecesariamente el código HTML de la página y complica el mantenimiento del código JavaScript. En general, este método sólo se utiliza para definir algunos eventos y en algunos otros casos especiales, como se verá más adelante.

1.3 Ubicación de las etiquetas script

Si utilizamos una etiqueta script para incluir el código JavaScript en nuestras páginas (tanto incorporado en la página como desde un fichero externo), conviene tener presente cuál es la mejor ubicación de dicho fichero.

- **Si el fichero sólo contiene funciones** que se van a activar o lanzar más adelante, a medida que interactuemos con la página, **podemos colocar este script en la cabecera (head).**
- **Si el fichero contiene instrucciones que necesitan ejecutarse inmediatamente** cuando la página se cargue, **entonces conviene incorporarlo al final de la página**, cuando ya se haya cargado todo el contenido de la misma:



1.4 La etiqueta noscript

Algunos navegadores no disponen de soporte completo de JavaScript, otros navegadores permiten bloquearlo parcialmente e incluso algunos usuarios bloquean completamente el uso de JavaScript porque creen que así navegan de forma más segura.

El lenguaje HTML define la etiqueta `<noscript>` para mostrar un mensaje al usuario cuando su navegador no puede ejecutar JavaScript. El siguiente código muestra un ejemplo del uso de la etiqueta `<noscript>`:

```
<head> ... </head>
<body>
  <noscript>
    <p>Bienvenido a Mi Sitio</p>
    <p>La página que estás viendo requiere para su funcionamiento el uso de JavaScript.
    Si lo has deshabilitado intencionadamente, por favor vuelve a activarlo.</p>
  </noscript>
</body>
```

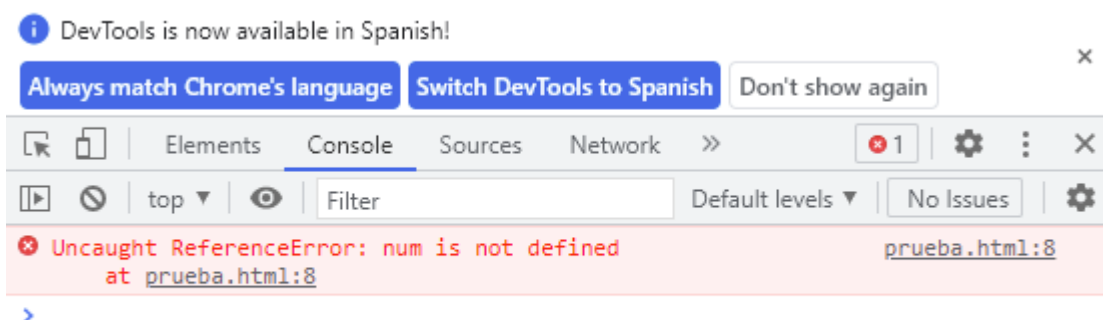
La etiqueta `<noscript>` se debe incluir en el interior de la etiqueta `<body>` (normalmente se incluye al principio de `<body>`). El mensaje que muestra `<noscript>` puede incluir cualquier elemento o etiqueta HTML.

1.5 Detectar errores en JavaScript

A diferencia de los lenguajes de programación de aplicaciones de escritorio, como Java o C#, en lenguajes de programación web como JavaScript o PHP es más difícil ver dónde está el error cuando una aplicación o página web no hace lo que debe, ya que no existe un terminal o consola accesible donde poder ver lo que el programa está haciendo.

La mayoría de navegadores proporcionan herramientas para poder depurar nuestro código JavaScript, y ver dónde están los errores cuando cargamos las páginas. Además, disponen de herramientas para revisar el código HTML y CSS y corregir posibles errores de diseño o estructura.

En Google Chrome, por ejemplo, con la tecla F12 (o haciendo clic derecho en la página y eligiendo la opción Inspeccionar) mostramos el panel de herramientas para desarrolladores. En él encontraremos varias secciones o pestañas. Una de las más útiles son la Consola, que muestra posibles errores en la ejecución del código JavaScript.





1.5.1 Consola y mensajes de error

Además de la instrucción `alert`, con la que podemos mostrar al usuario mensajes en la página a modo de popups, también podemos emplear la instrucción **`console.log`** o **`console.error`** para mostrar mensajes directamente en esta consola de depuración. Esto nos puede venir bien para dejar un registro de mensajes durante la ejecución del programa, y también para informar de errores que se puedan producir y que hayamos detectado.

```
console.error("El email está vacío");
```

1.6 Sintaxis

La sintaxis de un lenguaje de programación se define como el conjunto de reglas que deben seguirse al escribir el código fuente de los programas para considerarse como correctos para ese lenguaje de programación.

La sintaxis de JavaScript es muy similar a la de otros lenguajes de programación como Java y C.

Las normas básicas que definen la sintaxis de JavaScript son las siguientes:

- **No se tienen en cuenta los espacios en blanco y las nuevas líneas:** como sucede con HTML, el intérprete de JavaScript ignora cualquier espacio en blanco sobrante, por lo que el código se puede ordenar de forma adecuada para entenderlo mejor (tabulando las líneas, añadiendo espacios, creando nuevas líneas, etc.)
- **Se distinguen las mayúsculas y minúsculas:**
- **No se define el tipo de las variables:** al crear una variable, no es necesario indicar el tipo de dato que almacenará. De esta forma, una misma variable puede almacenar diferentes tipos de datos durante la ejecución del script.
- **No es necesario terminar cada sentencia con el carácter de punto y coma (;):** en la mayoría de lenguajes de programación, es obligatorio terminar cada sentencia con el carácter ;. Aunque JavaScript no obliga a hacerlo, es conveniente seguir la tradición de terminar cada sentencia con el carácter del punto y coma (;).
- **Se pueden incluir comentarios:** los comentarios se utilizan para añadir información en el código fuente del programa. JavaScript define dos tipos de comentarios: los de una sola línea y los que ocupan varias líneas.

Ejemplo de comentario de una sola línea:

```
// a continuación se muestra un mensaje  
alert("mensaje de prueba");
```

Los comentarios de una sola línea se definen añadiendo dos barras oblicuas (//) al principio de la línea.

Ejemplo de comentario de varias líneas:

```
/* Los comentarios de varias líneas son muy útiles cuando se necesita incluir bastante información  
en los comentarios */  
alert("mensaje de prueba");
```



Los comentarios multilínea se definen encerrando el texto del comentario entre los símbolos `/*` y `*/`.

1.7 El primer script

A continuación, se muestra un primer script sencillo pero completo:

```
<!DOCTYPE html>
<html >
  <head>
    <meta charset="UTF-8">
    <title>El primer script</title>
    <script type="text/javascript">
      alert("Hola Mundo!");
    </script>
  </head>
  <body>
    <p>Esta página contiene el primer script</p>
  </body>
</html>
```

La instrucción `alert()` es una de las utilidades que incluye JavaScript y permite mostrar un mensaje en la pantalla del usuario. Si se visualiza la página web de este primer script en cualquier navegador, automáticamente se mostrará una ventana con el mensaje "Hola Mundo!".

Ejercicio 1.1

Modificar el primer script para que:

1. Todo el código JavaScript se encuentre en un archivo externo llamado `codigo.js` y el script siga funcionando de la misma manera.
2. Después del primer mensaje, se debe mostrar otro mensaje que diga *"Soy el primer script"*
3. Añadir algunos comentarios que expliquen el funcionamiento del código
4. Añadir en la página HTML un mensaje de aviso para los navegadores que no tengan activado el soporte de JavaScript

1.8 Programación básica

Antes de comenzar a desarrollar programas y utilidades con JavaScript, es necesario conocer los elementos básicos con los que se construyen las aplicaciones. Si ya sabes programar en algún lenguaje de programación, este capítulo te servirá para conocer la sintaxis específica de JavaScript.

Si nunca has programado, este capítulo explica en detalle y comenzando desde cero los conocimientos básicos necesarios para poder entender posteriormente la programación avanzada, que es la que se utiliza para crear las aplicaciones reales.



1.8.1 Comunicación con el usuario

Ya hemos visto que con la instrucción **alert** podemos sacar mensajes por pantalla en un pequeño cuadro de diálogo, poniendo entre paréntesis el mensaje que queremos mostrar. Con las instrucciones `console.log` y `console.error` estos mensajes se muestran en la consola del navegador, que no está directamente visible o accesible por los usuarios:

```
alert("Hola, buenos días");  
  
console.log("El email está vacío");
```

Alternativamente, también podemos emplear la instrucción **document.write**, que muestra la información directamente en la página HTML donde se ejecuta el código JavaScript:

```
document.write("Esto es un texto en el HTML");
```

Otra instrucción útil para comunicarnos de forma básica con el usuario es `prompt`. Al igual que en el caso anterior, entre paréntesis podemos mostrar un mensaje, pero con esta instrucción también podemos permitir que el usuario escriba algo (en un cuadro de texto), recoger lo que ha escrito y guardarlo en una variable, para luego poderlo procesar y utilizar.

```
let nombre = prompt("Dime tu nombre");
```

Sacaría un cuadro para que el usuario escribiese su nombre, y al pulsar en Aceptar quedaría guardado en la variable `nombre`. Si finalmente el usuario cancela, la variable queda con un valor nulo (`null`).

1.8.2 Variables

Las variables en JavaScript se pueden crear de tres formas:

<code>let n1 = 5;</code>	Aconsejado , crea una variable local al bloque de código, (más adelante se verán las diferencias entre variables locales y globales)
<code>var n2 = 5;</code>	Desaconsejado , crea una variable global
<code>n3 = 5;</code>	Desaconsejado , crea una variable global

La palabra reservada **let** solamente se debe indicar al definir por primera vez la variable, lo que se denomina **declarar** una variable.

Si cuando se declara una variable se le asigna también un valor, se dice que la variable ha sido **inicializada**. En JavaScript no es obligatorio inicializar las variables, ya que se pueden declarar por una parte y asignarles un valor posteriormente.:

```
let numero_1;  
  
let numero_2;  
  
numero_1 = 3;  
  
numero_2 = 1;  
  
let resultado = numero_1 + numero_2;
```



El nombre de una variable también se conoce como **identificador** y debe cumplir las siguientes normas:

- Sólo puede estar formado por letras, números y los símbolos \$ (dólar) y _ (guion bajo).
- El primer carácter no puede ser un número.

Por tanto, las siguientes variables tienen nombres correctos:

```
let $numero1;  
  
let _$letra;  
  
let $$$otroNumero;  
  
let $_a_$4
```

Sin embargo, las siguientes variables tienen identificadores incorrectos:

```
let 1numero; // Empieza por un número  
  
let numero;1_123; // Contiene un carácter ";"
```

1.8.3 Tipos de variables

1.8.3.1 Numéricas

Se utilizan para almacenar valores numéricos enteros o decimales. En este caso, el valor se asigna indicando directamente el número entero o decimal. Los números decimales utilizan el carácter . (punto) en vez de , (coma) para separar la parte entera y la parte decimal:

```
let iva = 16; // variable tipo entero  
  
let total = 234.65; // variable tipo decimal
```

1.8.3.2 Cadenas de texto o strings

Se utilizan para almacenar caracteres, palabras y/o frases de texto. Para asignar el valor a la variable, se encierra el valor entre comillas dobles o simples, para delimitar su comienzo y su final:



```
let mensaje = "Bienvenido a nuestro sitio web";

let nombreProducto = 'Producto ABC';

let letraSeleccionada = 'c';
```

En ocasiones, el texto que se almacena en las variables no es tan sencillo. Si por ejemplo el propio texto contiene comillas simples o dobles, la estrategia que se sigue es la de encerrar el texto con las comillas (simples o dobles) que no utilice el texto:

```
let texto1 = "Una frase con 'comillas simples' dentro";

let texto2 = 'Una frase con "comillas dobles" dentro';
```

No obstante, a veces las cadenas de texto contienen tanto comillas simples como dobles. Además, existen otros caracteres que son difíciles de incluir en una variable de texto (tabulador, ENTER, etc.) Para resolver estos problemas, JavaScript define un mecanismo que consiste en sustituir el carácter problemático por una combinación simple de caracteres. A continuación se muestra la tabla de conversión que se debe utilizar:

Sí se quiere incluir...	Se debe incluir...
Una nueva línea	\n
Un tabulador	\t
Una comilla simple	\'
Una comilla doble	\"
Una barra inclinada	\\

De esta forma, el ejemplo anterior que contenía comillas simples y dobles dentro del texto se puede rehacer de la siguiente forma:

```
let texto1 = 'Una frase con \'comillas simples\' dentro';

let texto2 = "Una frase con \"comillas dobles\" dentro";
```

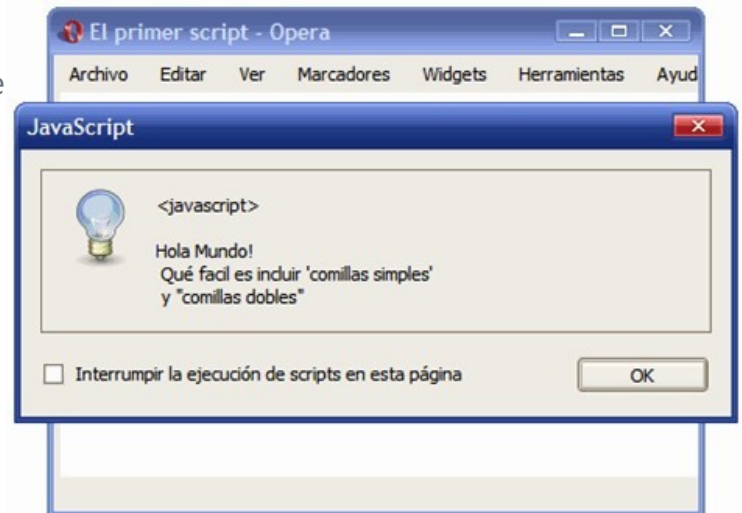
Este mecanismo de JavaScript se denomina "*mecanismo de escape*" de los caracteres problemáticos, y es habitual referirse a que los caracteres han sido "*escapados*".

Ejercicio 1.2

Crea un documento llamado **02_comillas.js.html** modificando el ejercicio anterior para que:



1. El mensaje que se muestra al usuario se almacene en una variable llamada mensaje y el funcionamiento del script sea el mismo.
2. El mensaje mostrado sea el de la siguiente imagen:



1.8.4 Booleanos

Las variables de tipo *boolean* o *booleano* también se conocen con el nombre de variables de tipo lógico.

Una variable de tipo *boolean* almacena un tipo especial de valor que solamente puede tomar dos valores: *true* (verdadero) o *false* (falso). No se puede utilizar para almacenar números y tampoco permite guardar cadenas de texto.

```
let clienteRegistrado = false;
let ivaIncluido = true;
```

1.8.5 Conversiones entre tipos simples

En ocasiones nos puede interesar convertir un tipo simple en otro. Por ejemplo, convertir un texto a entero, o un número en texto. Para ello JavaScript dispone de distintas instrucciones de conversión.

Por ejemplo, la instrucción `Number` permite convertir un valor a algo numérico. Podemos emplearlo para convertir textos a enteros o reales:

```
var texto = "32";
var numero = Number(texto); // 32
```

Alternativamente, también disponemos de las instrucciones `parseInt` y `parseFloat` para convertir a entero o real, respectivamente.

```
var texto = "32";
var numero = parseInt(texto); // 32
```

Para hacer el paso opuesto (convertir un número, o cualquier otro dato, a texto), podemos emplear la instrucción `String`:



```
var numero = 23;
var texto = String(numero); // "23"
```

Del mismo modo, también podemos emplear la instrucción `Boolean` para convertir un dato a booleano.

```
var texto = "true";
var resultado = Boolean(texto); // true
```

Una de las aplicaciones prácticas que tiene esto es poder convertir datos recogidos del usuario a otros tipos. Por ejemplo, ya hemos visto que con la instrucción `prompt` podemos pedirle al usuario que introduzca algún dato. Estos datos, JavaScript los interpreta directamente como texto. Así, si el usuario escribe "32", JavaScript no lo va a interpretar como que ha escrito el número 32, sino como que ha escrito los símbolos 3 y 2, simplemente.

Si queremos tratar lo que ha escrito el usuario como un número, para luego poder hacer operaciones con ese dato (sumarlo, por ejemplo), deberemos convertir el dato a numérico. El siguiente ejemplo calcula el doble del número que indique el usuario, usando `parseInt`.

```
var numero = prompt("Dime un número");
var doble = parseInt(numero) * 2;
alert("El doble es " + doble);
```

NaN

En algunas conversiones u operaciones debemos tener especial cuidado con que los datos con que trabajamos sean numéricos, ya que de lo contrario el resultado que se va a producir va a ser NaN (abreviatura de Not a Number). Por ejemplo, si sumamos un entero y un valor `undefined`, el resultado será NaN.

```
var numero;
var otroNumero = parseInt(numero);
// otroNumero es NaN porque numero es *undefined*
```

Disponemos de la instrucción `isNaN` para comprobar si un dato no es numérico (devuelve `true` cuando no lo es):

```
if (isNaN(valor))
{
    alert("El valor no es numérico");
}
```



Ejercicio 1.3

Crea un documento llamado **03_Operaciones_js.html** y, con JavaScript, pídele al usuario que introduzca dos números (primero uno y luego otro) y saca cuatro mensajes mostrando su suma, resta, multiplicación y división.

1.8.6 Operadores

Los operadores permiten manipular el valor de las variables, realizar operaciones matemáticas con sus valores y comparar diferentes variables. De esta forma, los operadores permiten a los programas realizar cálculos complejos y tomar decisiones lógicas en función de comparaciones y otros tipos de condiciones.

1.8.6.1 Asignación

Este operador se utiliza para guardar un valor específico en una variable. El símbolo utilizado es = (no confundir con el operador == que se utiliza para comparar y se verá más adelante)

```
let numero1 = 3;
```

A la izquierda del operador, siempre debe indicarse el nombre de una variable. A la derecha del operador, se pueden indicar variables, valores, condiciones lógicas, etc:

```
let numero1 = 3;

let numero2 = 4;

// Ahora, la variable numero1 vale 5
numero1 = 5;

// Ahora, la variable numero1 vale 4
numero1 = numero2;
```

1.8.6.2 Incremento y decremento ++ --

Estos dos operadores solamente son válidos para las variables numéricas y se utilizan para incrementar o decrementar en una unidad el valor de una variable y pueden ir delante o detrás de la variable con diferente resultado

```
let a = 1;

let b = 5;

console.log(a++); // Imprime 1 y incrementa a (2)

console.log(++a); // Incrementa a (3), e imprime 3
```



```
console.log(++a + ++b); // Incrementa a (4) y b (6). Suma (4+6), e imprime 10
console.log(a-- + --b); // Decrementa b (5). Suma (4+5). Imprime 9. Decrementa a (3)
```

1.8.6.3 Lógicos

El resultado de cualquier operación que utilice operadores lógicos siempre es un valor lógico o *booleano*.

1.8.6.4 Negación !

Uno de los operadores lógicos más utilizados es el de la negación. Se utiliza para obtener el valor contrario al valor de la variable:

```
let visible = true;
alert(!visible); // Muestra "false" y no "true"
```

La negación lógica se obtiene prefijando el símbolo ! al identificador de la variable. El funcionamiento de este operador se resume en la siguiente tabla:

variable	!variable
true	false
false	true

1.8.6.5 AND &&

La operación lógica AND obtiene su resultado combinando dos valores booleanos. El operador se indica mediante el símbolo && y su resultado solamente es true si los dos operandos son true:

variable1	variable2	variable1 && variable2
true	true	true
true	false	false
false	true	false
false	false	false

```
var valor1 = true;
var valor2 = false;
resultado = valor1 && valor2; // resultado = false

valor1 = true;
valor2 = true;
resultado = valor1 && valor2; // resultado = true
```



1.8.6.6 OR ||

La operación lógica OR también combina dos valores booleanos. El operador se indica mediante el símbolo || y su resultado es true si alguno de los dos operandos es true:

variable1	variable2	variable1 variable2
true	true	true
true	false	true
false	true	true
false	false	false

```
var valor1 = true;
var valor2 = false;
resultado = valor1 || valor2; // resultado = true

valor1 = false;
valor2 = false;
resultado = valor1 || valor2; // resultado = false
```

1.8.6.7 Aritméticos

- Los operadores aritméticos son: resta (-), multiplicación (*), división (/) y módulo (%).
- Estos operadores operan siempre con números, por tanto, cualquier operando que no sea un número debe ser convertido a número.

```
console.log(4 * 6); // Imprime 24

console.log("Hello " * "world!"); // Imprime NaN

console.log("24" / 12); // Imprime 2 (24 / 12)

console.log("42" * true); // Imprime 42 (42 * 1)

console.log("42" * false); // Imprime 0 (42 * 0)

console.log("42" * undefined); // Imprime NaN

console.log("42" - null); // Imprime 42 (42 - 0)

console.log(12 * "hello"); //Imprime NaN("hello" no puede ser convertido a número)

console.log(13 * 10 - "12"); // Imprime 118 ((13 * 10) - 12)
```

- el operador "módulo", calcula el resto de la división entera de dos números. Si se divide por ejemplo 10 y 5, la división es exacta y da un resultado de 2. Sin embargo, si se divide 9 y 5, la división no es exacta, el resultado es 1 y el resto 4, por lo que módulo de 9 y 5 es igual a 4.



```
let numero1 = 10; let numero2 = 5;

let resultado = numero1 % numero2; // resultado = 0

numero1 = 9; numero2 = 5;

resultado = numero1 % numero2; // resultado = 4
```

Los operadores matemáticos también se pueden combinar con el operador de asignación para abreviar su notación:

```
let numero1 = 5;

numero1 += 3; // numero1 = numero1 + 3 = 8

numero1 -= 1; // numero1 = numero1 - 1 = 4

numero1 *= 2; // numero1 = numero1 * 2 = 10

numero1 /= 5; // numero1 = numero1 / 5 = 1

numero1 %= 4; // numero1 = numero1 % 4 = 1
```

1.8.6.8 Relacionales

- Estos operadores son: >, <, <=, >=, ==, ===, !=, !==
- Los operadores de comparación, comparan dos valores y devuelven un booleano (true o false)
- Estos operadores son prácticamente los mismos que en la mayoría de lenguajes de programación, a excepción de algunos, que veremos a continuación.
- Podemos usar == o === para comparar la igualdad (o lo contrario !=, !==).
 - La principal diferencia es que el primero, no tiene en cuenta los tipos de datos que están siendo comparados, compara si los valores son equivalentes.
- Cuando usamos ===, los valores además deben ser del mismo tipo.
 - Si el tipo de dato o el valor son diferentes devolverá falso.
 - Devolverá true cuando ambos valores son idénticos y del mismo tipo.

```
console.log(3 == "3"); // true

console.log(3 === "3"); // false

console.log(3 != "3"); // false
```



```
console.log(3 !== "3"); // true

console.log(false == null); // false (null no es equivalente a cualquier boolean).

console.log(false == undefined); // false (undefined no es equivalente a cualquier boolean).

console.log(null == undefined); // true (regla especial de JavaScript)

console.log(0 == false); // true

console.log({} >= false); // Object vacío -> false

console.log([] >= false); // Array vacío -> true
```

1.9 Estructuras de control de flujo

Si se utilizan estructuras de control de flujo, los programas dejan de ser una sucesión lineal de instrucciones para convertirse en programas inteligentes que pueden tomar decisiones en función del valor de las variables.

1.9.1 Estructura if

La estructura más utilizada en JavaScript y en la mayoría de lenguajes de programación es la estructura if. Se emplea para tomar decisiones en función de una condición. Su definición formal es:

```
if (condición) {
    ....
}
```

Si la condición se cumple (es decir, si su valor es true) se ejecutan todas las instrucciones que se encuentran dentro de {...}. y el programa continúa ejecutando el resto de instrucciones del script.

Ejemplo:

```
let mostrarMensaje = true;

if(mostrarMensaje == true) {
    alert("Hola Mundo");
}
```



La comparación del ejemplo anterior suele ser el origen de muchos errores de programación, al confundir los operadores `==` y `=`. Las comparaciones siempre se realizan con el operador `==`, ya que el operador `=` solamente asigna valores:

```
// Error - Se asigna el valor "false" a la variable

if(mostrarMensaje = false) {
    ...
}
```

Los operadores AND y OR permiten encadenar varias condiciones simples para construir condiciones complejas:

```
var mostrado = false;

var usuarioPermiteMensajes = true;

if(!mostrado && usuarioPermiteMensajes) {
    alert("Es la primera vez que se muestra el mensaje");
}
```

1.9.2 Estructura if...else

```
if(condicion) {

    ...

}else {

    ...

}
```

Si la condición se cumple (es decir, si su valor es `true`) se ejecutan todas las instrucciones que se encuentran dentro del `if()`. Si la condición no se cumple (es decir, si su valor es `false`) se ejecutan todas las instrucciones contenidas en `else {}`.

Ejemplo

```
let price = 65;

if(price < 50) {
    console.log("Esto es barato!");
} else if (price < 100) {
    console.log("Esto no tan barato...");
} else {
```



```
    console.log("Esto es caro!");  
}
```

1.9.3 Estructura switch

- Similar a otros lenguajes de programación.
- Se evalúa una variable y se ejecuta el bloque correspondiente al valor que tiene (puede ser numero, string,...).
- Al final de cada bloque pondremos la instrucción break, ya que, de no ponerlo continuaría ejecutando las instrucciones que haya en el siguiente bloque.
- Ejemplo en el que dos valores ejecutaran el mismo bloque de código:

```
let userType = 1;  
switch(userType) {  
    case 1:  
    case 2: // Tipos 1 y 2 entran aquí  
        console.log("Puedes acceder a esta zona");  
        break;  
    case 3:  
        console.log("No tienes permisos para acceder aquí");  
        break;  
    default: // Ninguno de los anteriores  
        console.error("Tipo de usuario erróneo!");  
}
```

Ejercicio 1.4

Crea un documento llamado **04_calculoDNI.js.html** y, con JavaScript realiza lo siguiente: El cálculo de la letra del Documento Nacional de Identidad (DNI) es un proceso matemático sencillo que se basa en obtener el resto de la división entera del número de DNI y el número 23. A partir del resto de la división, se obtiene la letra seleccionándola dentro de un array de letras.

El array de letras es:

```
let letras = ['T', 'R', 'W', 'A', 'G', 'M', 'Y', 'F', 'P', 'D', 'X', 'B', 'N', 'J', 'Z', 'S', 'Q', 'V', 'H', 'L', 'C', 'K', 'E', 'I'];
```

Por tanto si el resto de la división es 0, la letra del DNI es la T y si el resto es 3 la letra es la A. Con estos datos, elaborar un pequeño script que:

1. Almacene en una variable el número de DNI indicado por el usuario y en otra variable la letra del DNI que se ha indicado. (Pista: si se quiere pedir directamente al usuario que indique su número y su letra, se puede utilizar la función prompt())



2. En primer lugar (y en una sola instrucción) se debe comprobar si el número es menor que 0 o mayor que 99999999. Si ese es el caso, se muestra un mensaje al usuario indicando que el número proporcionado no es válido y el programa no muestra más mensajes.
3. Si el número es válido, se calcula la letra que le corresponde según el método explicado anteriormente.
4. Una vez calculada la letra, se debe comparar con la letra indicada por el usuario. Si no coinciden, se muestra un mensaje al usuario diciéndole que la letra que ha indicado no es correcta. En otro caso, se muestra un mensaje indicando que el número y la letra de DNI son correctos.

1.9.4 Estructura for

Crea un bucle que consiste en tres expresiones opcionales, encerradas en paréntesis y separadas por puntos y comas, seguidas de una sentencia ejecutada en un bucle.

`for ([expresion-inicial]; [condicion]; [expresion-final]) sentencia`

```
// ejemplo de bucle for
let limit = 5;

for (let i = 1; i <= limit; i++) { // Imprime 1 2 3 4 5
  console.log(i);
}
```

La idea del funcionamiento de un bucle for es la siguiente: "mientras la condición indicada se siga cumpliendo, repite la ejecución de las instrucciones definidas dentro del for. Además, después de cada repetición, actualiza el valor de las variables que se utilizan en la condición".

- La "inicialización" es la zona en la que se establece los valores iniciales de las variables que controlan la repetición
- La "condición" es el único elemento que decide si continua o se detiene la repetición.
- La "actualización" es el nuevo valor que se asigna después de cada repetición a las variables que controlan la repetición

```
// otro ejemplo con un array

let dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"];

for(let i = 0; i < 7; i++) {
  Console.log(dias[i]);
}
```

1.9.5 Estructura While

La estructura while permite crear bucles que se ejecutan ninguna o más veces, dependiendo de la condición indicada:



```
let value = 1;

while (value <= 5) { // Imprime 1 2 3 4 5
    console.log(value++);
}
```

Si la condición no se cumple ni siquiera la primera vez, el bucle no se ejecuta. Si la condición se cumple, se ejecutan las instrucciones una vez y se vuelve a comprobar la condición. Si se sigue cumpliendo la condición, se vuelve a ejecutar el bucle y así se continúa hasta que la condición no se cumpla.

Las variables que controlan la condición deben modificarse dentro del propio bucle, ya que de otra forma, la condición se cumpliría siempre y el bucle `while` se repetiría indefinidamente.

El programa debe sumar todos los números menores o igual que otro dado. Por ejemplo si el número es 5, se debe calcular: $1 + 2 + 3 + 4 + 5 = 15$

```
let resultado = 0;
numero = 5;
let i = 0;

while(i <= numero) {
    resultado += i;
    i++;
}

alert(resultado);
```

1.9.6 estructura do...while

El bucle de tipo `do...while` es muy similar al bucle `while`, salvo que en este caso **siempre** se ejecutan las instrucciones del bucle al menos la primera vez.

```
let value = 1;

do { // Imprime 1 2 3 4 5
    console.log(value++);
} while (value <= 5);
```

De esta forma, como la condición se comprueba después de cada repetición, la primera vez siempre se ejecutan las instrucciones del bucle. Es importante no olvidar que después del `while()` se debe añadir el carácter `;` (al contrario de lo que sucede con el bucle `while` simple).

Utilizando este bucle se puede calcular fácilmente el factorial de un número: el factorial de 5 por ejemplo es $5 \times 4 \times 3 \times 2 \times 1 = 120$.

```
let resultado = 1; let numero = 5;
```



```
do {  
    resultado *= numero; // resultado = resultado * numero  
    numero--;  
} while(numero > 0);  
  
alert(resultado);
```

Como en cada repetición se decrementa el valor de la variable `numero` y la condición es que `numero` sea mayor que cero, en la repetición en la que `numero` valga `0`, la condición ya no se cumple y el programa se sale del bucle `do...while`.

1.9.7 Sentencias `break` y `continue`

Las sentencias `break` y `continue` permiten manipular el comportamiento normal de los bucles para detener el bucle o para saltarse algunas repeticiones. Concretamente, la sentencia `break` permite terminar de forma abrupta un bucle y la sentencia `continue` permite saltarse algunas repeticiones del bucle.

El siguiente ejemplo muestra el uso de la sentencia `break`:

```
let value = 1;  
  
do {  
    // Imprime 1 2  
    if (value >= 3) {  
        break;  
    }  
    console.log(value++);  
} while (value <= 5);
```

Si el programa llega a una instrucción de tipo `break`, sale inmediatamente del bucle y continúa ejecutando el resto de instrucciones que se encuentran fuera del bucle `for`.

En ocasiones, lo que se desea es saltarse alguna repetición del bucle cuando se dan algunas condiciones.

```
let value = 1;  
  
do {  
    // Imprime 1 2 4 5  
    if (value == 3) {  
        continue;  
    }  
    console.log(value++);  
} while (value <= 5);
```

En este caso, cuando `value == 3` no se termina el bucle, sino que no se ejecutan las instrucciones de esa repetición y se pasa directamente a la siguiente repetición del bucle `for`.



Ejercicio 1.5

Crea un documento llamado **05_sumaAcumulada_js.html** y, con JavaScript, calcula y muestre la suma acumulada del 1 al 5 es decir: $1+2+3+4+5 = 15$

utilizando:

- la estructura `for(;;){...}`
- La estructura `while {...}`
- la estructura `do{...} while.`

Ejercicio 1.6

Crea un documento llamado **06_factorial_js.html**. El factorial de un número entero n es una operación matemática que consiste en multiplicar todos los factores $n \times (n-1) \times (n-2) \times \dots \times 1$. Así, el factorial de 5 (escrito como $5!$) es igual a: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Realiza un script que calcule el factorial de 5 utilizando:

- la estructura `for(;;){...}`
- La estructura `while {...}`
- la estructura `do{...} while.`



2 - Tema 2 - Funciones

Si ponemos cualquier fragmento de código JavaScript de los que hemos visto hasta ahora en una página, se ejecutará nada más cargar la página, y eso no es lo habitual. Lo normal es que el código JavaScript se vaya activando o ejecutando a medida que el usuario vaya haciendo cosas en la página. Por ejemplo, cuando el usuario pulse el botón de enviar un formulario, debería activarse el código que comprueba que es correcto.

Para evitar la ejecución inmediata del código JavaScript, y de paso poderlo estructurar mejor y dejarlo preparado para que cada trozo de código se active en un momento determinado, usamos funciones. Las funciones son fragmentos de código a los que les ponemos un nombre, de forma que podemos elegir el momento en que ejecutar esas instrucciones simplemente utilizando el nombre que hayamos puesto a la función.

2.1 Definición de funciones

Estas funciones se crean con la palabra `function` seguida del nombre que queramos y unos paréntesis. El nombre que pongamos a la función debe ser representativo de lo que hace, y normalmente sólo usaremos letras, números y subrayados(`_`), como si fueran nombres de variables. Después, entre llaves, pondremos todas las instrucciones que queremos que se ejecuten en bloque. Por ejemplo, esta función crea dos variables numéricas, las suma y muestra el resultado:

```
function sumaVariables()  
{  
    var num1 = 2;  
    var num2 = 6;  
    var suma = num1 + num2;  
    alert("El resultado de la suma es " + suma);  
}
```

Si luego queremos hacer que este código se active al hacer clic en un párrafo, por ejemplo, pondríamos esto:

```
<p onclick="sumaVariables()">Bla bla bla bla...</p>
```

Como hemos comentado, el orden en que definamos las funciones en el código de la página (o del archivo JavaScript) es irrelevante. Podemos ponerlas antes o después del bloque de código que las invoca.

2.1.1 Uso de parámetros

Entre los paréntesis de las funciones podemos introducir, separados por comas, unos datos llamados parámetros. Básicamente son variables, a los que se les da valor desde otro lugar (por



ejemplo, el contenido principal de la página) y se pueden utilizar dentro de la función. Esta función, por ejemplo, suma los dos datos que recibe como parámetros y muestra el resultado.

```
function sumaDatos(num1, num2)
{
    var suma = num1 + num2;
    alert("El resultado de la suma es " + suma);
}
```

Podríamos llamarla desde otra función, o desde el contenido de la página, con algo como esto:

```
<p onclick="sumaDatos(3, 5)">Bla bla bla bla...</p>
```

NOTA: es habitual en JavaScript que tanto los nombres de funciones como los nombres de parámetros (y variables) sigan la nomenclatura camel case: comenzar en minúscula y luego iniciar cada palabra del nombre de la variable en mayúscula: sumaDatos, nombreUsuario, etc.

Ejercicio 2.1

Crea un documento llamado **01_operaciones_funcion_js.html**. En él, define cuatro párrafos: uno que ponga "Suma", otro que ponga "Resta", otro que ponga "Multiplicación" y otro que ponga "División". Añade un código JavaScript que defina cuatro funciones llamadas sumar, restar, multiplicar y dividir que reciban dos números como parámetro y hagan cada una de estas cuatro operaciones con los números que reciben, y muestren su resultado, como en el ejemplo anterior. Estas funciones se deben activar respectivamente cuando hagamos clic sobre el párrafo correspondiente, es decir, al hacer clic en el párrafo que pone "Suma", deberemos activar la función sumar, y así sucesivamente. Pasa como parámetro a las funciones los números que quieras.

2.1.2 Número de parámetros y parámetros por defecto

JavaScript es algo laxo en cuanto al paso de parámetros a funciones, de forma que no "se queja" si pasamos parámetros de más o de menos. Si una función necesita dos parámetros y le pasamos sólo el primero, el segundo tomará el valor undefined. Si enviamos parámetros de más, el resto se descarta.

Además, podemos dar valores por defecto a los parámetros que lo necesiten, asignándoselo en la declaración de la función:

```
function sumaDatos(num1, num2 = 0)
{
    var suma = num1 + num2;
    alert("El resultado de la suma es " + suma);
}
```



Según la declaración del ejemplo anterior, si llamamos a la función con `sumaDatos(3, 2)` obtendrá un resultado de 5, y si la llamamos con `sumaDatos(3)` obtendrá un resultado de 3, porque al segundo parámetro se le asigna el valor por defecto de 0.

Es importante que los parámetros que tengan valor por defecto se coloquen al final de la secuencia de parámetros, para así poner primero todos los que tienen que tener valor, y después los que pueden no tenerlo, y no alterar el orden inicial de los parámetros en la declaración.

2.1.3 Tipo de retorno

En ocasiones nos interesa que una función produzca o devuelva un resultado, para poderlo utilizar en alguna otra expresión, o almacenarlo en una variable. Por ejemplo, la función de suma anterior podría devolver el resultado de la suma en lugar de mostrarlo con un alert, y así nosotros poder decidir qué hacer con ese valor (sacarlo por pantalla, guardarlo en un campo de formulario...).

Para indicar qué valor devuelve una función, empleamos la instrucción `return` seguida del valor que se quiere devolver. Nuestra función de suma anterior podríamos transformarla en esta otra:

```
function sumaDatos(num1, num2)
{
    var suma = num1 + num2;
    return suma;
}
```

NOTA: también podríamos haber puesto directamente `return num1 + num2` y ahorrarnos la variable adicional `suma`.

En este ejemplo almacenamos el resultado en una variable, y luego la mostramos por la consola.

```
let resultado = sumaDatos(3, 5);
console.log(resultado);
```

Ejercicio 2.2

Crea una página llamada **02_calcular_precio.js.html** que defina una función llamada `calcularPrecio` con 3 parámetros: el nombre de un producto, su precio y su porcentaje de IVA.

- Los tres parámetros deberán tener un valor por defecto: "Producto genérico", 100 y 21.
- Internamente, se debe convertir el nombre del producto a string y los otros dos datos a números
- Si el precio o el porcentaje no son numéricos, se mostrará el mensaje de "Datos incorrectos"



- Si son válidos, mostraremos por consola el nombre del producto y el precio final incluyendo el IVA
- Llama a la función varias veces con varios datos de prueba para comprobar su funcionamiento

Ejercicio 2.3:

Crea una página llamada **03_tabla_multiplicar_js.html**. Deberá pedirle al usuario un número del 1 al 10 (repetidamente hasta que sea válido) y después, usando una función previamente definida, mostrar la tabla de multiplicar de ese número en el contenido de la página (`document.write`), en forma de lista no ordenada.

2.1.4 Variables locales y globales

Una variable definida dentro de una función es local a la función (no existe fuera de ella), independientemente de si se ha declarado con `var` o con `let`. Una variable declarada fuera de cualquier función es global a todas ellas. Cuando existe una variable local con el mismo nombre que una global, prevalece la local sobre la global. Dicho de otro modo, JavaScript comienza a buscar las variables en el ámbito local, y si no las encuentra, pasa al ámbito global.

Modo estricto

Si en algún momento usamos una variable que no hemos declarado, se considera una variable global a toda la aplicación. Esto puede provocar importantes efectos colaterales en un programa, y no es algo deseable.

Para evitar que JavaScript sea tan permisivo, podemos habilitar su modo estricto, incluyendo esta instrucción al principio del fichero o código JavaScript:

```
'use strict';
```

2.2 Algunas funciones útiles

Además de poder definir nuestras propias funciones, JavaScript incorpora una serie de funciones que nos pueden resultar útiles para ciertos ámbitos, como por ejemplo el manejo de cadenas de texto, o algunos cálculos matemáticos, entre otras categorías.

2.2.1 Funciones para manejo de textos

Ya vimos anteriormente que uno de los tipos básicos que maneja JavaScript son los textos, que se podían representar indistintamente entre comillas dobles o simples.

```
let texto = "Hola, buenas";
```



```
let otroTexto = 'Otro texto distinto';
```

Además, el propio objeto o variable que almacena el texto dispone de una serie de instrucciones útiles que nos permiten manipularlo o transformarlo. Veremos aquí algunas de las instrucciones más útiles.

- El operador `+` se emplea para concatenar cadenas de texto, enlazando una a continuación de otra.
- Para acceder a un carácter determinado de la cadena, se utilizan los **corchetes** con su posición, empezando en 0. También se puede utilizar la instrucción `charAt` indicando entre paréntesis la posición a obtener (también empezando por la 0).
- La propiedad `length` devuelve el tamaño o número de caracteres de la cadena en cuestión.
- La función `indexOf` sirve para buscar un texto dentro de la cadena, devolviendo la posición donde se encuentra, o -1 si no hay coincidencias. También se puede emplear `includes` para ver si un texto está contenido dentro de otro.
- En el caso de querer buscar patrones, podemos emplear la función `search`, que busca la primera ocurrencia que encaje con un patrón.
- Las funciones `endsWith` y `startsWith` determinan si un texto termina o empieza por otro, respectivamente.
- Las funciones `toUpperCase` y `toLowerCase` convierten a mayúscula y minúscula, respectivamente, la cadena entera.
- La función `replace` se emplea para reemplazar un texto con otro. Recibe como parámetros el texto viejo y el nuevo, y reemplaza sólo la primera ocurrencia encontrada.
- Para obtener una subcadena a partir de otra, empleamos la instrucción `substring`. Como parámetros indicamos desde qué posición empezar a obtener la subcadena, y hasta qué posición llegar (esta última posición NO se incluye en la subcadena). Si no se indica este segundo dato, se obtiene hasta el final de la cadena. Alternativamente, se puede usar la función `slice`, con los mismos parámetros y utilidad, y también se puede emplear la función `substr`, que se diferencia de las anteriores en que el segundo parámetro no indica la posición hasta la que llegar, sino cuántos caracteres queremos obtener desde la posición inicial.
- La función `split` sirve para partir una cadena por un delimitador indicado, obteniendo un array con todas las partes encontradas.
- La función `trim` (sin parámetros) elimina posibles espacios adicionales que pueda haber al principio o al final de la cadena. Es especialmente útil cuando se recoge información de un formulario, donde el usuario puede haber introducido espacios accidentalmente.
- Para comparar dos cadenas podemos emplear los operadores de comparación `==`, `!=`, `>`, `<`. Hay que tener en cuenta que esta comparación es alfabética.



Veamos un ejemplo de todas estas funciones. En comentarios se indica el resultado que se produce en cada paso.

```
let texto = "Hola";
texto = texto + " buenas";           // texto = "Hola buenas"
console.log(texto[0]);               // H
console.log(texto.charAt(0));        // H
let tamaño = texto.length;          // 11
let posición = texto.indexOf("buenas"); // 5
let contiene = texto.includes("bue"); // true
let posición2 = texto.search(/[aeiou]/); // 1 (posición de la primera vocal minúscula)
let mayus = texto.toUpperCase();     // mayus = "HOLA BUENAS"
let texto2 = texto.replace("bu", "BU"); // texto2 = "HoLa BUenas"
let subcad = texto.substring(5, 7);  // subcad = "bu"
let subcad2 = texto.substr(5, 3);    // subcad2 = "bue"
let partes = texto.split(" ");       // partes = ["Hola", "buenas"]
let menor = texto < "Adiós";         // false
let empieza = texto.startsWith("Hola"); // true
```

Ejercicio 2.4:

Crea una página llamada **04_ejercicio_strings_js.html** con las siguientes funciones:

- Función que reciba como parámetro dos textos t1 y t2 y devuelva el texto resultado de quitar de t1 todas las apariciones de t2
- Función que reciba como parámetro un nombre de fichero (texto) y determine si es una imagen JPG o PNG.
- Función que reciba como parámetro un texto y determine si es una matrícula válida. Una matrícula se considera válida si está compuesta por 4 dígitos y 3 letras mayúsculas.

Comprueba el funcionamiento de todas estas funciones desde el propio código JavaScript, con los datos de entrada que quieras.

2.2.2 Funciones matemáticas

A través del elemento Math de JavaScript podemos acceder a una serie de funciones matemáticas, y también a algunas constantes numéricas que pueden resultar útiles, como el número PI o el número E.

- Math.PI y Math.E representan estos dos valores numéricos (pi y el número e)
- Math.round(x) redondea x al entero más cercano (por arriba o por abajo)
- Math.floor(x) y Math.ceil(x) aproximan al entero más cercano por abajo o por arriba, respectivamente.



- `Math.min(n1, n2, n3...)` y `Math.max(n1, n2, n3...)` obtienen el menor/mayor valor de los que se le pasan como parámetro, respectivamente.
- `Math.pow(b, e)` obtiene la potencia de elevar una base `b` a un exponente `e`.
- `Math.abs(x)` obtiene el valor absoluto de `x`
- `Math.random()` genera un número real aleatorio entre 0 y 1 (sin contar el 1)
- `Math.sqrt(x)` calcula la raíz cuadrada de `x`

Veamos algunos ejemplos de uso:

```
let n1 = Math.sqrt(16);           // 4
let n2 = Math.min(2, 6, 1, 7);    // 1
let n3 = Math.pow(3, 2);          // 9
let n4 = Math.ceil(3.12);         // 4
let n5 = Math.abs(-3);            // 3
```

2.2.3 Trabajando con fechas

Para gestionar fechas JavaScript proporciona el elemento `Date`. Podemos crear fechas de distintas formas:

```
let fecha1 = new Date();           // Fecha y hora actuales
let fecha2 = new Date(2021, 3, 4); // 4 de abril de 2021
let fecha3 = new Date("2021-04-04"); // 4 de abril de 2021
let fecha4 = Date.now();           // Fecha actual en milisegundos desde el 1/1/1970

let texto1 = fecha2.toLocaleDateString(); // 4/4/2021
```

2.3 Opciones avanzadas con funciones

Para terminar este documento, veamos algunas opciones algo más avanzadas que se pueden llevar a cabo con las funciones.

2.3.1 Funciones anónimas

Las funciones anónimas, como su propio nombre indica, son funciones que no tienen nombre. Al declararlas con la palabra `function` no se les asigna ningún nombre, y lo normal es asignar su contenido a una variable para poderlas usar más adelante, o bien definir las en el momento en que se van a usar.

Por ejemplo, la siguiente función devuelve la suma de sus dos parámetros, pero esta vez la hemos definido como una función anónima asignada a la variable `suma`:



```
let suma = function(n1, n2) {  
    return n1 + n2;  
}  
  
...  
console.log(suma(4, 3));    // 7
```

A priori, usar una función anónima no aporta muchas ventajas respecto a una función tradicional. Se suelen emplear en el caso de funciones de un solo uso (cuando definimos una función que no se va a usar en ninguna otra parte del código), para poder definir el código de la función justo en el punto donde se va a usar.

2.3.2 Funciones lambda o arrow functions

Las funciones lambda son una forma abreviada de definir funciones, disponible desde ES2015. La idea es definir una función anónima pero con otra sintaxis:

- Primero indicamos los parámetros de la función, entre paréntesis y separados por comas. En el caso de que sólo haya un parámetro, se pueden omitir los paréntesis
- Después añadimos una flecha => (por eso se llaman arrow functions o funciones flecha)
- Después colocamos, entre llaves, el código de la función.

La función anónima de suma anterior podríamos expresarla así:

```
let suma = (n1, n2) => {  
    return n1 + n2;  
}
```

Como particularidad de las funciones lambda, si la función que implementamos se limita a devolver un valor, podemos omitir las llaves y la palabra return. El ejemplo anterior quedaría así de corto:

```
let suma = (n1, n2) => n1 + n2;
```

Entre las ventajas del uso de las funciones lambda, conseguimos un código más compacto y resumido. Además, son ampliamente usadas en algunos ámbitos, como en la programación en servidor usando el framework Node.js. Igual que ocurre con las funciones anónimas, pueden utilizarse en el mismo lugar en que se definen.



Ejercicio 2.5:

Crea una página llamada **05_ejercicio_lambdas.js.html** donde repites el ejercicio anterior definiendo las funciones requeridas como funciones lambda.

2.3.3 Cargar diversos archivos fuente

Además de definir funciones para modularizar nuestro código JavaScript, cuando la aplicación es lo suficientemente compleja, o cuando utilizamos librerías externas como las que veremos más adelante, puede ser necesario dividir el código en varios archivos fuente, de forma que cada uno de ellos incorpora un conjunto de funciones o utilidades. Podemos añadir así diversas etiquetas script en nuestro código HTML para incorporar cada uno de estos ficheros:

```
<!DOCTYPE html>
<html lang="es">
  ...
  <script src="fichero1.js"></script>
  <script src="fichero2.js"></script>
  ...
</html>
```

En esta situación, pueden darse dos casos:

- Las funcionalidades incorporadas por todos los archivos son independientes o, dicho de otro modo, no hay dos funciones que se llamen igual en ningún archivo. En este caso no hay ningún problema. Dependiendo de la función que utilicemos en un punto determinado del HTML, se invocará desde un archivo o desde otro.
- Las funcionalidades de los archivos se solapan (hay funciones con el mismo nombre en algunos archivos). En este caso, prevalecerá la función del último archivo incorporado. Por ejemplo, si en el caso anterior ambos archivos fichero1.js y fichero2.js tienen una función llamada prueba entonces se aplicará la de fichero2.js (aunque tengan distinto número de parámetros una y otra)

Ejercicio 2.6

Crea una página llamada **06_par_impar.js.html** y escribe el código de una función a la que se pasa como parámetro un número entero y devuelve como resultado una cadena de texto que indica si el número es par o impar. Mostrar por pantalla el resultado devuelto por la función.

Ejemplo:

document.write(esParImpar(5)) → devuelve: "5 es impar"



`document.write(esParImpar(6) → devuelve: "6 es par"`

Ejercicio 2.7

Crea una página llamada **07_mayus_minus_js.html** define una función que muestre información sobre una cadena de texto que se le pasa como argumento. A partir de la cadena que se le pasa, la función determina si esa cadena está formada sólo por mayúsculas, sólo por minúsculas o por una mezcla de ambas.

Ejemplo:

`document.write(mayusculasMinusculas("Hola Mundo")) → devuelve: "Mayúsculas y minúsculas"`

`document.write(mayusculasMinusculas("HOLA MUNDO")) → devuelve: "Mayúsculas"`

`document.write(mayusculasMinusculas("hola mundo")) → devuelve: "Minúsculas"`

Ejercicio 2.8

Crea una página llamada **08_palindromo_js.html** define una función que determine si la cadena de texto que se le pasa como parámetro es un palíndromo, es decir, si se lee de la misma forma desde la izquierda y desde la derecha. Ejemplo de palíndromo complejo: "La ruta nos aporó otro paso natural".

Ejemplo:

`document.write(palindromo("ana")) → devuelve: "ana es palindromo"`

`document.write(palindromo("anas")) → devuelve: "anas no es palindromo"`

Ejercicio 2.9

Crea una página llamada **09_tabla_multiplicar_js.html**

- Crea un script (usando al menos 1 función) en el que se le pida al usuario 2 datos:
 - El primero serán dos números del 1 al 10 (debes dar error en otro caso)
 - El segundo será el modo de visualización: LISTA o TABLA (debe dar error si elige otra cosa) puedes mostrarlo con la instrucción `document.write()`;
- El programa mostrará la tabla de multiplicar del número más pequeño, llegando hasta el número más grande
- Ojo, tendrás que tener en cuenta que el usuario puede ponerte el segundo número más pequeño que el primero.



- Ejemplo, si el usuario introduce los número 2 y 4, el resultado de la página será el siguiente

2x1	=	2
2x2	=	4
2x3	=	6
2x4	=	8

Modo tabla

- 2x1=2
- 2x2=4
- 2x3=6
- 2x4=8

Modo lista

Ejemlo:

let desde, hasta, modo

//..pedir los datos y filtrar..

tablaMultiplicar(desde,hasta,modo);

Ejercicio 2.10

Crea una página llamada **10_ejercicio_strings_js.html** d

1. Crea una función que reciba un string con un path linux (directorios separados por /) y devuelva un array que contenga todos los nombres de los directorios del path en sus elementos (valora utilizar split)

ejemplo:

let path="./usr/bin/temp";

Console.log(pathObtenerCarpetas(path)) → devolverá [".","usr","bin","temp"]

- Crea una función que reciba dos strings y devuelva otro string que contenga el primero con todas las apariciones del segundo string eliminadas. La búsqueda debe ser case insensitive, es decir, no se distinguirá entre mayúsculas y minúsculas.

Se debe utilizar la función string.replace(patron, reemplazo)

ejemplo:

let texto="mi padre se llama Manolo y yo me llamo Manolo";

let patron = "Manolo" //texto a sustituir

console.log(reemplazar(texto,patron))

// el resultado final será -> "mi padre se llama y yo me llamo"



3 Tema 3 - Gestión de arrays y datos complejos

Un array, en cualquier lenguaje de programación, es una variable que sirve para almacenar varios datos dentro. En muchos lenguajes estos arrays tienen un tamaño fijo e inamovible durante la ejecución del programa, pero en JavaScript no (de hecho, no son arrays propiamente dichos, sino vectores o listas).

Normalmente los datos que se almacenan en un array son del mismo tipo (por ejemplo, un conjunto de números enteros, o un conjunto de cadenas de texto), aunque en el caso de JavaScript y otros lenguajes (como PHP), cada elemento de la lista o conjunto puede ser de un tipo diferente al resto.

3.1 Manipulación básica de arrays en JavaScript

Para crear arrays en JavaScript, creamos la variable que va a almacenar el conjunto con dos corchetes o bien con `new Array`. En este último caso, podemos indicar el tamaño inicial del array también:

```
let conjunto = [];  
let conjunto2 = new Array();  
let conjunto3 = new Array(10);
```

También podemos definir a la vez el array y los elementos iniciales que va a tener en una misma línea (aunque luego podemos añadir más o cambiar los que hay).

```
let conjunto = [10, "Hola", 3.5];  
let conjunto2 = new Array(10, "Hola", 3.5);
```

Después, podemos ir añadiendo elementos a esta colección usando los corchetes y poniendo dentro qué posición ocupa el elemento en la colección, empezando por la posición 0. Por ejemplo:

```
conjunto[0] = 10;  
conjunto[1] = "Hola";  
conjunto[2] = 3.5;  
...
```



Alternativamente, también podemos emplear la instrucción `push` para añadir elementos al final de los existentes:

```
conjunto.push(3);  
conjunto.push("Buenas");
```

Observemos cómo cada posición guarda un dato distinto (un entero, un texto, un número real...). Esto es posible, como decimos, en ciertos lenguajes de programación, especialmente los de tipado dinámico, es decir, aquellos donde no hay que indicar de qué tipo es una variable, sino que toma el tipo del valor que se le asigna. Si volvemos a utilizar una misma posición, borraremos el elemento previo y asignaremos el nuevo, pudiendo también cambiar el tipo de dato de esa posición:

```
conjunto[1] = "Adiós"; // Ahora este elemento ya no vale "Hola"
```

Si utilizamos una posición no correlativa con las anteriores, se añadirá un elemento en esa posición, y quedarán huecos vacíos enmedio, a los que se les asigna el valor `undefined`.

3.1.1 Arrays y referencias

Si asignamos a una variable simple el valor de otra, estamos creando una copia de la variable original, con lo que si modificamos la segunda variable, la primera no ve su valor alterado:

```
let a = 3;  
let b = a;  
b = 4;  
alert(a); // 3
```

Esto no es así con los arrays, ya que almacenan un conjunto mayor de datos. Cuando asignamos una variable array a otra, estamos asignando una referencia a la posición de memoria del array original, con lo que cualquier cambio en esa segunda variable va a afectar al array original:

```
let datos = [1, 2, 3, 4];  
let datos2 = datos;  
datos2[0] = 10;  
console.log(datos); // [10, 2, 3, 4]
```



3.2 2. Recorrido de arrays

En ocasiones es posible que nos toque recorrer una lista o array de elementos de una web, buscando uno en concreto. Para ello, disponemos de las estructuras repetitivas o bucles vistos en documentos anteriores (while, do..while o for). Por ejemplo, supongamos que tenemos una variable lista con un array de elementos (el que sea). Si queremos recorrerlo podemos utilizar una variable que vaya desde el principio del array (posición 0) hasta el final (indicado por la propiedad length del array):

```
for (let i = 0; i < lista.length; i++)
{
    alert (lista[i]);    // Muestra el valor de cada elemento
}
```

Otra alternativa es utilizar una variable que sirva de índice en la lista, tomando el valor de cada posición de la misma, y accediendo con dicho valor a la posición concreta del array:

```
for (let indice in lista)
{
    alert (lista[indice]); // Muestra el valor de cada elemento
}
```

Una tercera alternativa que podemos emplear desde ES2015 es el bucle for..of, que permite iterar por todos los valores del array (y también por todas las letras de un texto):

```
for (let valor of lista)
{
    alert(valor);
}
```

Sobre la propiedad length, se cuentan las posiciones hasta la última ocupada, aunque en medio haya posiciones no definidas (undefined). Por ejemplo, este array tiene tamaño 5:

```
let a = [];
a[4] = 3;
```

3.3 Operaciones con arrays

Pasamos a detallar ahora algunas de las operaciones más útiles que podemos realizar con arrays:

- **push** nos sirve para añadir elementos al final de los disponibles en un array. Podemos pasarle tantos como queramos, separados por comas.
- **unshift** añade elementos al principio del array. También podemos pasarle los que queramos, separados por comas.
- **pop** devuelve y elimina el último elemento del array.



- **shift** devuelve y elimina la primera posición.
- **Split** une todos los elementos de un array para formar una cadena de texto. Para unir los elementos se utiliza el carácter separador indicado
- **join** se aplica sobre un array. Puede recibir como parámetro un delimitador (texto) y obtiene un texto separando todos los elementos del array con el delimitador.
- **concat** enlaza dos arrays, obteniendo otro con la unión de todos los elementos de ambos
- **slice** obtiene un subarray desde una posición inicial (incluida) hasta una final (no incluida)
- **splice** quita elementos de un array, indicando desde qué posición eliminar y cuántos elementos quitar desde ahí. Adicionalmente, admite más parámetros, que serían los datos a añadir en la posición donde nos hemos quedado.
- **reverse** invierte el orden de los elementos de un array
- **indexOf** recibe como parámetro un dato, y nos devuelve en qué posición del array se encuentra por primera vez, o -1 si no se encuentra.
- **includes** recibe como parámetro un dato y devuelve si dicho dato está en el array o no.

Veamos aquí un ejemplo de uso de cada una de estas funciones:

```
let datos = [1, 2, 3, 4];
datos.unshift(5, 6);           // [5, 6, 1, 2, 3, 4]
let variable = datos.pop();     // variable = 4, datos = [5, 6, 1, 2, 3]
let texto = datos.join(":");    // texto = "5:6:1:2:3"
texto.split(":");              // [5, 6, 1, 2, 3]
let datos2 = datos.concat([10, 9]); // [5, 6, 1, 2, 3, 10, 9]
let datos3 = datos.slice(1, 3);  // [6, 1]
datos2.splice(1, 3, 20, 30);     // [5, 20, 30, 3, 10, 9]
datos2.reverse();               // [9, 10, 3, 30, 20, 5]
let posicion = datos2.indexOf(3); // 2
let existe = datos2.includes(10); // true
```

Ejercicio 1:

Realiza los siguientes pasos en un fichero llamado **01_arrays_js.html**. Muestra por consola el resultado después de aplicar cada uno, pero con los elementos separados por "=>" usando join:

- Crea un array con 4 elementos numéricos
- Concatena 2 elementos más al final y 2 al principio (unshift y push)
- Elimina las posiciones de la 3 a la 5 (incluida) (splice)
- Inserta 2 elementos más entre el penúltimo y el último (splice)
- Invierte el array (reverse)



3.4 Ordenación de arrays

La función `sort` permite ordenar el array sobre el que se aplica. Dicha ordenación suele ser alfabética (por ejemplo, `40 < 5`, o `"Adiós" < "Hola"`).

```
let nombres = ["Juan", "Ana", "Laura", "Blas"];
nombres.sort(); // ["Ana", "Blas", "Juan", "Laura"]
```

Sin embargo, podemos pasar como parámetro a la función `sort` una función anónima que defina un criterio de comparación alternativo. Dicha función recibe dos parámetros que actúan como dos datos cualesquiera del array, y devuelve un número que será:

- Negativo si el primer parámetro debe ir antes que el segundo
- Positivo si el primer parámetro debe ir después que el segundo
- Cero si ambos parámetros son iguales, y deben ir uno junto al otro en la ordenación

El siguiente ejemplo ordena ascendentemente (en orden numérico) un array de enteros:

```
let datos = [20, 4, 6, 2, 10];
datos.sort(function(n1, n2) {
  if (n1 < n2)
    return -1;
  else if (n1 > n2)
    return 1;
  else
    return 0;
});
```

Alternativamente, podemos expresarla de esta otra forma más resumida, con el mismo resultado:

```
let datos = [20, 4, 6, 2, 10];
datos.sort(function(n1, n2) {
  return n1 - n2;
});
```

Ejercicio 2:

Crea un fichero llamado **02_arrays_ordenacion_js.html**. Crea dentro un array de números y muéstralo ordenado de mayor a menor.



3.4.1 Otras operaciones avanzadas con arrays

Existen otras operaciones que podemos aplicar sobre arrays que tienen un uso un poco más avanzado. Muchas de ellas reciben una función como parámetro, que se aplica a cada elemento del array.

La función `every` obtiene un booleano indicando si la función que recibe como parámetro es cierta para todos los elementos del array. El siguiente ejemplo comprueba si todos los elementos del array son pares:

```
let resultado = datos.every(function(n) {  
    return n % 2 == 0;  
});
```

La función `some` es similar a la anterior, pero devuelve un booleano indicando si alguno de los elementos del array cumple la función. Este ejemplo comprueba si alguno de los números del array es mayor que 10:

```
let resultado = datos.some(function(n) {  
    return n > 10;  
});
```

La función `forEach` recorre cada elemento del array, aplicándole la función que recibe como parámetro. Este ejemplo muestra por consola cada elemento del array:

```
datos.forEach(function(n) {  
    console.log(n);  
});
```

La función `filter` obtiene como resultado un array con los elementos que cumplen la condición indicada por la función que se pasa como parámetro. El siguiente ejemplo obtiene un subconjunto del array inicial con los datos que sean mayores que 10:

```
let datosMayores10 = datos.filter(function(n){  
    return n > 10;  
});
```

La función `map` permite aplicar una transformación (definida por la función recibida como parámetro) a todos los elementos de un array. El siguiente ejemplo multiplica por 2 cada elemento del array:

```
let dobles = datos.map(function(n) {  
    return n * 2;  
});
```



```
});
```

Notar que en todas estas funciones podemos emplear arrow functions o funciones lambda como parámetro:

```
let resultado = datos.every(n => n % 2 == 0);  
let datosMayores10 = datos.filter(n => n > 10);  
let dobles = datos.map(n => n * 2);
```

Ejercicio 3:

Crea un fichero llamado **03_funciones_arrays_js.html**. Define una función que reciba un array y un número como parámetros y devuelva si todos los números del array son múltiplos del número recibido como parámetro. Haz que el número tome por defecto el valor de 2, y prueba la función con distintas llamadas.

Ejercicio 4:

Crea un fichero llamado **04_funciones_arrays2_js.html**. Define un array de datos numéricos, quédate sólo con los números que sean positivos y transforma el array para que almacene los cuadrados de esos números. Usa funciones lambda para resolver este ejercicio.

3.5 Objetos JavaScript

Aunque JavaScript no es un lenguaje orientado a objetos propiamente dicho, sí que permite definir clases (veremos algún ejemplo de ello más adelante) y trabajar con objetos que almacenen información heterogénea.

Para definir un objeto JavaScript, encerramos sus datos entre llaves. Cada propiedad que queramos definir llevará un nombre y, después de los dos puntos, le podemos dar un valor inicial. Así podríamos definir los datos de una persona:

```
let persona = {  
  nombre: "Nacho",  
  edad: 43,  
  telefono: "611223344"  
};  
  
console.log(persona.nombre);
```

Como podemos comprobar, una vez hemos definido los datos de la persona u objeto, accedemos a ellos con el operador punto ., separando el nombre de la variable y el del campo a consultar. También podemos añadir sobre la marcha nuevos campos sobre el objeto ya creado:

```
let persona = {  
  nombre: "Nacho",  
  edad: 43,
```



```
telefono: "611223344"  
};  
persona.direccion = "C/Mayor 13";
```

Ejercicio 5:

Crea una página llamada **05_Objetos_js.html**. Define un array con datos de personas como los del ejemplo anterior. Ordena después el array ascendentemente por el nombre de las personas (alfabético), recórrelo y muéstralo por pantalla. Puede serte de utilidad la función `localeCompare` que permite comparar alfabéticamente dos textos y obtener un entero indicando cuál es mayor o menor.

Ejercicio 6

Dado un array de números enteros y utilizando `array.every()` crea una función anónima que devuelva cierto si todos los elementos son pares y falso en caso contrario.

Ejercicio 7

Crea una función que reciba un array con nombres de alumnos y muestre por pantalla los nombres ordenados alfabéticamente y numerados.

Puedes utilizar `forEach` para recorrer el array

Ejercicio 8

Crea una función que reciba un array y devuelva otro solo con sus elementos pares.

Puedes utilizar `filter`

Ejercicio 9

Crea una función que reciba un array y un valor, e indique si dicho valor se encuentra en el array o no. Si existe, devolverá la posición del elemento, si no, devolverá -1.

Ejemplo:

```
let array=["uno","dos","tres"];  
let valor="cuatro"  
Console.log(arrayContieneValor(array,valor)) → devolverá -1  
let valor ="dos"  
Console.log(arrayContieneValor(array,valor)) → devolverá 1
```



4 Tema 4 - Acceso a los elementos de un documento DOM

4.1 Introducción. BOM y DOM

JavaScript es un lenguaje que se ejecuta normalmente en un navegador, sobre una página web. Esta característica hace que JavaScript pueda acceder directamente a ciertas propiedades del navegador y de la propia página sobre la que se está ejecutando, lo que se conoce como BOM (Browser Object Model) y DOM (Document Object Model), respectivamente.

En esta sección veremos algunas de las propiedades más relevantes que pueden ser accedidas y utilizadas desde JavaScript en uno y otro modelo. Por ejemplo, cambiar la URL actual del navegador, o acceder a un elemento concreto de un documento para comprobar su valor o para cambiarlo.

4.2 Elementos principales del BOM

Dentro del conjunto de elementos definidos por el navegador o BOM, el elemento principal que lo engloba todo es el elemento `window` (ventana). A partir de este elemento podemos hacer tareas como:

- Obtener las dimensiones de la pantalla donde estamos
- Mostrar mensajes (instrucciones `alert` o `prompt` que ya hemos utilizado antes)
- Cambiar la URL de la página actual, para ir a otra página distinta
- ... etc.

Desde el propio elemento `window` podemos acceder a otros subelementos para completar estas tareas. Por ejemplo:

- La pantalla (elemento `screen`), a partir de la cual podemos acceder a ciertos datos como la anchura y altura, profundidad de color, etc.
- El navegador en sí (elemento `navigator`), donde podemos acceder al nombre del navegador, plugins instalados...
- El documento que tenemos abierto (elemento `document`), nos va a permitir acceder a la URL del documento, su título, enlaces...



- La ubicación del documento actual (elemento `location`) nos permitirá centrarnos en la URL del documento actual, pudiéndola cambiar para cargar otras páginas.
- El historial de navegación (elemento `history`) con el que podremos ir hacia adelante o atrás en el mismo.

Veamos algunos ejemplos de uso básico de algunas de estas propiedades:

```
// Sacamos por consola el ancho y alto de la pantalla:
console.log(window.screen.width + " x " + window.screen.height);

// Cargamos otra página
window.location.href = "https://iessanvicente.com";
// También sirve
location.href = "https://iessanvicente.com";

// Vamos a la página anterior del historial de navegación
window.history.back();
// También sirve
history.back();
```

4.2.1 Uso de timeouts e intervalos

Una de las aplicaciones más útiles que podemos extraer del uso del BOM es la posibilidad de definir **timeouts**, un tiempo pasado el cual queremos que se ejecute una instrucción o conjunto de instrucciones. Para ello usamos la función `timeout` indicándole qué función queremos ejecutar, y cuántos milisegundos queremos que pasen antes de ejecutarla. Por ejemplo, el siguiente código saca por consola la fecha actual pasados 3 segundos (usamos una arrow function para definir la función a ejecutar):

```
setTimeout(() => console.log(new Date()), 3000);
```

Una opción un poco más elaborada consist en definir **intervalos**. La diferencia con lo anterior es que la función no sólo se ejecuta una vez, sino que se ejecuta periódicamente tras cada intervalo de tiempo definido. Este código muestra la fecha por consola cada 5 segundos:

```
setInterval(() => console.log(new Date()), 5000);
```

Podemos también cancelar los intervalos que tengamos habilitados en cualquier momento. Para ello, debemos obtener el id del intervalo generado cuando lo creamos, y luego llamar a la función `clearInterval` con ese id. Por ejemplo, este código activa el intervalo anterior y, pasados 20 segundos, lo para.



```
let idIntervalo = setInterval(() => console.log(new Date()), 5000);
setTimeout(() => clearInterval(idIntervalo), 20000);
```

Tanto a `setTimeout` como a `setInterval` podemos pasarles una función ya existente, no es necesario definirla sobre la marcha con una arrow function. Si la función necesita parámetros adicionales, podemos pasárselos después del tiempo de espera. El siguiente timeout saluda al nombre que se le pasa como parámetro, pasados 5 segundos:

```
function saludar(nombre)
{
    console.log("Hola, " + nombre);
}
setTimeout(saludar, 5000, "Nacho");
```

4.3 Uso del DOM

A través del modelo del documento o DOM podemos acceder a la estructura de información contenida en dicho documento: etiquetas, subetiquetas, atributos, etc. Esto nos puede servir para:

- Consultar información de ciertos elementos de la página. Por ejemplo, validar si los datos de un formulario son correctos, o cuántos elementos de un tipo hay en una página
- Modificar la información de ciertos elementos. Por ejemplo, cambiar sus propiedades CSS (color de fondo, visibilidad), o añadir/quitar elementos dinámicamente en la misma página.

Para acceder o manipular la estructura del documento utilizaremos el objeto `document`. Es un objeto global que podemos utilizar directamente, o bien desde el objeto `window` que hemos visto antes (`window.document`, aunque es mucho más habitual utilizar `document` a secas).

4.3.1 Instrucciones para explorar el DOM

Las siguientes instrucciones nos pueden resultar muy útiles para explorar la estructura del DOM:

- `document.documentElement` obtiene el elemento raíz (html)
- `document.head` obtiene la cabecera (elemento head)
- `document.body` obtiene el cuerpo (elemento body)
- `document.getElementById("id")` obtiene el elemento cuyo id hemos indicado como parámetro



- `document.getElementsByClassName("clase")` obtiene el conjunto de elementos de la clase indicada.
- `document.getElementsByTagName("etiqueta")` obtiene el conjunto de etiquetas del tipo indicado.
- `element.children` obtiene todas las etiquetas que cuelgan del elemento actual
- `element.parentNode` obtiene la etiqueta padre de la actual
- `element.nextSibling` obtiene el siguiente hermano del elemento actual. También tenemos disponible `element.previousSibling`. Alternativamente, tenemos `nextElementSibling` o `previousElementSibling` si sólo queremos recorrer etiquetas, y no nodos de texto.
- `element.innerHTML` obtiene el contenido HTML del elemento actual. Alternativamente, podemos emplear la propiedad `textContent` o `innerText` para mostrar el contenido textual (sin subetiquetas, ya que las renderiza) que tenga el elemento.

Podemos llamar a estas instrucciones (especialmente a las que buscan por id, clase o nombre de etiqueta) desde el documento en general (`document`) o desde un elemento en particular, con lo que buscará sólo en el interior de ese documento.

Veamos cómo utilizar estas instrucciones con el siguiente ejemplo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Prueba DOM</title>
</head>
<body>
  <h1>Ejemplo de prueba del DOM</h1>
  <p>Párrafo de prueba</p>
  <ul id="lista">
    <li>Uno</li>
    <li>Dos</li>
    <li>Tres</li>
  </ul>
  
</body>
</html>
```

Sobre este ejemplo HTML, ejecutamos las siguientes instrucciones JavaScript. Deberemos ejecutarlas al final del documento, cuando ya se haya cargado:

```
let lista = document.getElementById("lista");
let items = lista.getElementsByTagName("li");
alert(items.length); // 3
```



```
for(let i = 0; i < items.length; i++)
{
    // Uno, Dos, Tres, respectivamente
    alert(items[i].textContent);
}
let imagen = document.getElementById("imagen1");
// Ruta hasta imagen "imagen.png"
alert(imagen.src);
```

Ejercicio 1:

Crea un documento HTML llamado **01_pruebaDOM.html**, con este contenido (cópialo y pégalo):

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Prueba con el DOM</title>
</head>
<body>
<p>Lorem ipsum dolor sit amet, <a href="http://prueba">consectetuer
adipiscing elit</a>. Sed mattis enim vitae orci. Phasellus libero.
Maecenas nisl arcu, consequat congue, commodo nec, commodo ultricies,
turpis. Quisque sapien nunc, posuere vitae, rutrum et, luctus at,
pede. Pellentesque massa ante, ornare id, aliquam vitae, ultrices
porttitor, pede. Nullam sit amet nisl elementum elit convallis
malesuada. Phasellus magna sem, semper quis, faucibus ut, rhoncus non,
mi. <a href="http://prueba2">Fusce porta</a>.</p>

<p>Aenean at nisl. Maecenas egestas dapibus odio. Vestibulum ante ipsum
primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin
consequat auctor diam. <a href="http://prueba">Ut bibendum blandit est</a>.
Curabitur vestibulum. Nunc malesuada porttitor sapien. Aenean a lacus et
metus venenatis porta. Suspendisse cursus, sem non dapibus tincidunt,
lorem magna porttitor felis, id sodales dolor dolor sed urna. Sed rutrum
nulla vitae tellus. Sed quis eros nec lectus tempor lacinia. Aliquam nec
lectus nec neque aliquet dictum. Etiam <a href="http://prueba3">consequat
sem quis massa</a>.</p>

<p id="p3">Vestibulum aliquet, nulla sit amet imperdiet suscipit, nunc
erat laoreet est, a <a href="http://prueba">aliquam leo odio sed sem</a>.
Quisque eget eros vehicula diam euismod tristique. Proin
<a href="http://prueba4">egestas</a> adipiscing ligula. Duis iaculis
laoreet turpis. Mauris mollis est sit amet diam. Curabitur hendrerit,
eros quis malesuada tristique, ipsum odio euismod tortor, a vestibulum
nisl mi at odio. <a href="http://prueba5">Sed non lectus non est
```



```
pellentesque</a> auctor.</p>
```

```
</body>
```

```
</html>
```

Después, añade un bloque de código JavaScript donde, utilizando los métodos de acceso al DOM vistos aquí, muestres con mensajes (alert) la siguiente información:

- Cuántos enlaces tiene la página en total
- A qué dirección enlaza el primer enlace (deberás acceder a su propiedad href)
- Cuántos enlaces hay en el tercer párrafo en total

Observa también que, para que el código Javascript funcione en este ejemplo, deberemos añadirlo al final de la página, ya que de lo contrario aún no se habrán cargado los contenidos (sólo el bloque head), y no tendrá nada que examinar al ejecutarse.

4.3.2 Uso del query selector

Con la aparición de la librería jQuery en el año 2006, algunas de las operaciones anteriores se pudieron expresar de forma más simple gracias a la sintaxis incorporada por el propio lenguaje. Así, acceder a un elemento por su id era tan sencillo como hacer `$("#id")`. Es por ello que, con el tiempo, el propio lenguaje JavaScript incorporó el uso de esta sintaxis, a través del método `querySelector` del documento. Veamos algunos ejemplos:

- `document.querySelector("a")` obtendría el primer enlace de la página
- `document.querySelector(".miClase")` obtendría el primer elemento de clase `miClase`
- `document.querySelector("#miID")` obtendría el elemento con el id `miID`
- `document.querySelector("h1, h2")` obtendría el primer encabezado `h1` o `h2`
- `document.querySelector("div > p")` obtendría el primer párrafo que fuera hijo inmediato de un `div`
- ...

Como alternativa al método `querySelector`, también disponemos de `querySelectorAll`, que obtiene una lista (array) con todos los elementos que cumplan el patrón indicado. Por ejemplo, `document.querySelectorAll("a")` obtendrá un array con todos los enlaces del documento. Así podríamos obtenerlos y mostrar sus títulos (atributo `title`):

```
let enlaces = document.querySelectorAll("a");
enlaces.forEach(elem => console.log(elem.title));
```

Ejercicio 2:

Crea un documento HTML llamado **02_pruebaDOM2.html**, con el mismo contenido que el anterior, y trata de resolverlo usando el query selector esta vez.



4.3.2.1 Búsquedas de elementos y recorrido en bucles

A la hora de utilizar las instrucciones `getElementXXXX` o el query selector hay que tener en cuenta algunas diferencias importantes. Por ejemplo, el tipo de colección que se devuelve con uno u otro es distinto, y eso hace que no se pueda utilizar la instrucción `forEach` si empleamos las instrucciones `getElementXXXX`:

```
let parrafos = document.getElementsByTagName("p");
parrafos.forEach(p => { ... });    // ERROR
```

En cambio, sí podemos recorrerlo con un `for` tradicional:

```
let parrafos = document.getElementsByTagName("p");
for (let p of parrafos) { ... }    // OK
```

Si empleamos el query selector, podremos utilizar uno u otro tipo de recorrido indistintamente:

```
let parrafos = document.querySelectorAll("p");
parrafos.forEach(p => { ... });    // OK
for (let p of parrafos) { ... }    // OK
```

4.3.3 Manipulación del DOM

Una vez sabemos acceder a los elementos del documento, podemos modificar el contenido del mismo: añadir nuevos elementos, borrar elementos existentes o modificar el contenido de los que hay.

4.3.3.1 Crear contenidos

Para crear contenido, seguiremos estos pasos:

1. Crearemos un objeto del tipo que queramos añadir (párrafo, `h1`, `div`, etc.)
2. Crearemos el contenido de dicho objeto
3. Añadiremos el contenido al objeto
4. Añadiremos el objeto al documento, en la posición que queramos

Por ejemplo, si queremos añadir un párrafo con el texto "Hola" al final del documento, el código JavaScript sería el siguiente:

```
// 1. Creamos el párrafo
```



```
let parrafo = document.createElement("p");
// 2. Creamos su contenido
let contenido = document.createTextNode("Hola");
// 3. Añadimos el contenido
parrafo.appendChild(contenido);
// 4. Añadimos el párrafo
document.body.appendChild(parrafo);
```

En el caso de que quisiéramos añadir este mismo párrafo al final de un div con id = div1, primero localizaríamos este div, y luego le añadiríamos el párrafo a él, en lugar de al body:

```
// 0. Buscamos el lugar
let div = document.getElementById("div1");
// 1. Creamos el párrafo
let parrafo = document.createElement("p");
// 2. Creamos su contenido
let contenido = document.createTextNode("Hola");
// 3. Añadimos el contenido
parrafo.appendChild(contenido);
// 4. Añadimos el párrafo
div.appendChild(parrafo);
```

Alternativamente, también podemos emplear la instrucción `insertBefore` para añadir un elemento antes de otro. Por ejemplo, este código añade un nuevo item en una lista, justo delante del tercer elemento. Además, cambia el contenido de ese elemento para indicar que ahora es el cuarto:

```
<ul id="lista">
  <li>Primero</li>
  <li>Segundo</li>
  <li>Tercero</li>
</ul>
<script>
  let lista = document.getElementById("lista");
  let item3 = lista.children[2];
  let nuevoItem = document.createElement("li");
  nuevoItem.innerText = "Nuevo tercero";
  lista.insertBefore(nuevoItem, item3);
  item3.innerText = "Nuevo cuarto";
</script>
```



4.3.3.2 Modificar contenido existente

Si queremos modificar el contenido de un elemento existente, podemos acudir a la propiedad `innerHTML` de dicho elemento (una vez encontrado), o también la propiedad `innerText` (si sólo queremos cambiar el texto, sin subetiquetas), y le cambiamos el valor al del nuevo contenido. Por ejemplo, si queremos cambiar el contenido del párrafo con `id="miParrafo"`:

```
let parrafo = document.getElementById("miParrafo");
parrafo.innerHTML = "Nuevo <strong>contenido</strong> del párrafo";
```

4.3.3.3 Borrar contenido

Para borrar un contenido de la página, deberemos localizar el elemento (párrafo, `h1`, etc.) que queremos borrar, acceder a su padre, y decirle que borre a ese hijo. Por ejemplo, si queremos borrar un `div` cuyo `id` es `div1`:

```
let div = document.getElementById("div1");
div.parentNode.removeChild(div);
```

Ejercicio 3:

Copia el ejercicio **02_pruebaDOM.html** en un archivo llamado **03_pruebaDOM.html**, añadiendo las siguientes funcionalidades:

- Añadir un nuevo párrafo al final del documento que diga "Nuevo párrafo".
- Borrar el segundo párrafo del documento

4.3.3.4. Acceder a atributos concretos

En ocasiones nos puede interesar acceder a un atributo concreto de un elemento para ver o modificar su valor. Por ejemplo, si queremos acceder al atributo `class` de un párrafo cuyo `id` es `p1`:

```
let parrafo = document.getElementById("p1");
let claseParrafo = parrafo.className;
```

En general, los nombres de atributos coinciden con los que hayamos puesto en el HTML (`style`, `href`...) salvo algunas excepciones como `className`. Algunos de ellos no dejan modificar su valor (sólo leerlo), pero otros sí.

Caso práctico: acceso a los estilos CSS



Una de las principales utilidades del acceso a los atributos es poder acceder y modificar algunos estilos de la página en tiempo real. Por ejemplo, cambiar la anchura del párrafo del ejemplo anterior (width), o la visibilidad (display), para mostrarlo u ocultarlo:

```
parrafo.style.width="50%";  
parrafo.style.display="none";
```

Normalmente, para acceder a las propiedades CSS (como width, display...) se accede a la propiedad `style`, seguida del atributo o propiedad a modificar, que suele tener el mismo nombre que tienen en CSS; en el caso de que la propiedad tenga un nombre compuesto en CSS (background-color, font-weight...), se quita el guión y se pone la primera letra de la siguiente palabra en mayúsculas en la propiedad JavaScript (backgroundColor, fontWeight).

Este ejemplo cambia el color de fondo del párrafo anterior, y lo establece en rojo (observa que el valor de la propiedad siempre va entre comillas):

```
parrafo.style.backgroundColor="red";
```

Ejercicio 4:

Crea otra versión del ejercicio **03_pruebaDOM.html** anterior en un archivo **04_pruebaDOM.html**. Añade estos cambios sobre la versión inicial:

- Pon el color de fondo del primer párrafo en amarillo.
- Oculta el segundo párrafo

Ejercicio 5

Completar el código JavaScript proporcionado para que cuando se pinche sobre el enlace se muestre completo el contenido del texto. Además, el enlace debe dejar de mostrarse después de pulsarlo por primera vez. La acción de pinchar sobre un enlace forma parte de los "Eventos" de JavaScript que se ven en el siguiente capítulo. En este ejercicio, sólo se debe saber que al pinchar sobre el enlace, se ejecuta la función llamada `muestra()`.

Ejercicio 6

Completar el código JavaScript proporcionado para que se añadan nuevos elementos a la lista cada vez que se pulsa sobre el botón. Utilizar las funciones DOM para crear nuevos nodos y añadirlos a la lista existente. Al igual que sucede en el ejercicio anterior, la acción de pinchar sobre un botón forma parte de los "Eventos" de JavaScript que se ven en el siguiente capítulo. En este ejercicio, sólo se debe saber que al pinchar sobre el botón, se ejecuta la función llamada `anyade()`.



5 Tema 5 - Gestión de eventos

5.1 Eventos y programación tradicional

Los programas tradicionales como los que hemos hecho hasta ahora se limitan a ejecutar una secuencia concreta de instrucciones, una tras otra. Pero para ciertos tipos de aplicaciones esto puede no ser suficiente. Por ejemplo, podemos necesitar que nuestra aplicación haga algo cuando pulsamos un botón, o cuando pasamos el ratón por encima de una imagen. Es aquí donde debemos acudir a la programación dirigida por eventos (event driven programming).

Un evento, en general, es algo que sucede y que permite, a raíz de él, desencadenar una serie de acciones. Por ejemplo, cuando pulsamos el botón de cerrar en una ventana del ordenador, se produce un evento que ocasiona el cierre del programa en el que estamos. Cuando elegimos el menú Archivo > Guardar como... de una aplicación, se produce un evento que permite mostrar un cuadro de diálogo para elegir dónde queremos guardar lo que estamos haciendo.

Los eventos de JavaScript están a la espera de que suceda algo en la página web. Por ejemplo, que el usuario haga clic sobre algún elemento, o que pase el ratón por encima de alguna zona, o incluso eventos que no tienen que ver con acciones del usuario, como que el contenido de la página termine de cargarse. Una vez se produce un evento, podemos preparar a nuestro programa o aplicación web para que responda con una acción determinada. Para ello, JavaScript permite asignar una función a cada uno de los posibles eventos que pueden producirse en una página, sobre cada elemento.

5.2 Principales tipos de eventos

Sobre una página web se pueden producir diferentes tipos de eventos. Veremos a continuación algunos de los más importantes, indicando sobre qué elementos se pueden producir.

5.2.1 Eventos sobre el contenido de la página (body)

Los siguientes eventos se producen sobre el contenido general de la página:

- **load**: se produce cuando la página termina de cargarse. Podemos emplearlo, por ejemplo, para hacer ciertas operaciones sobre el DOM, una vez sabemos que el contenido ya está cargado.
- **unload**: se produce cuando se cierra la página.



- **beforeunload**: se produce antes de cerrar la página. Es típico utilizarlo en páginas donde se pregunta al usuario si está seguro que quiere salir, porque hay cambios que no se hayan guardado.
- **resize**: se produce cuando cambia el tamaño de la ventana donde está el contenido

5.2.2 Eventos de teclado

Los siguientes eventos se producen con acciones desde el teclado sobre la página:

- **keydown** / **keyup**: se producen cuando se pulsa y suelta una tecla, respectivamente
- **keypress**: se produce cuando se pulsa y levanta una tecla (ambas acciones juntas), aunque está desaconsejado (deprecated) en versiones recientes de JavaScript, y se recomienda usar uno de los dos anteriores.

5.2.3 Eventos de ratón

Estos eventos se producen utilizando el ratón sobre la página:

- **click** / **dblclick**: cuando hacemos clic o doble clic con cualquier botón del ratón, respectivamente
- **mousedown** / **mouseup**: cuando pulsamos o levantamos un botón del ratón, respectivamente
- **mouseenter**: cuando entramos con el ratón en una zona
- **mouseleave**: cuando salimos con el ratón de una zona
- **mousemove**: cuando movemos el ratón dentro de una zona

5.2.4 Eventos de formulario

Estos eventos se aplican sobre elementos de formularios:

- **focus**: cuando un elemento del formulario tiene el foco. Por ejemplo, un cuadro de texto en el que acabamos de entrar
- **blur**: cuando un elemento del formulario pierde el foco
- **change**: cuando el contenido del elemento ha cambiado. Por ejemplo, al cambiar el texto de un cuadro de texto, o la opción seleccionada en una lista. En el caso de cuadros de texto también se puede usar el evento **input**.
- **submit**: cuando se envía el formulario



5.2.5 Otros eventos

Existen otros muchos eventos a los que podemos responder: abortar la carga de una imagen, eventos sobre pantallas táctiles, etc. Podéis consultar una lista más detallada [aquí](#).

5.3 Uso básico de eventos

Como hemos visto, cada etiqueta o elemento de la página puede admitir un conjunto determinado de eventos, y responder ante ellos. Los eventos debemos asociarlos a funciones JavaScript para que puedan responder al mismo, y estas funciones se llaman manejadores de eventos (event handlers). Podemos definir estos manejadores de distintas formas.

5.3.1 Manejadores de eventos mediante atributos

Una primera forma de definir los manejadores de eventos consiste en utilizar una serie de atributos XHTML asociados a cada evento, y definir dentro el código JavaScript que queramos. Estos atributos tienen el mismo nombre del evento, con el prefijo "on" delante. Por ejemplo, así advertiríamos de que el usuario sale de un formulario:

```
<form action="pagina.php" onmouseleave="alert('Saliendo del formulario');">
```

Es una alternativa poco recomendable, porque se mezcla demasiado el código JavaScript y el HTML, y es difícilmente mantenible. Como valor añadido, podemos utilizar el elemento **this** para acceder al elemento que ha provocado el evento. Este código cambia el color de fondo de un div a amarillo cuando entramos en él:

```
<div onmouseenter="this.style.backgroundColor='yellow';">...</div>
```

5.3.2 Manejadores de eventos mediante atributos y funciones

Una segunda forma algo más cómoda con respecto a la anterior consiste en utilizar el atributo XHTML para invocar a una función externa que haga el trabajo. De este modo, si el evento requiere de varios pasos complejos, no tenemos que ponerlos todos dentro del atributo, y el código JavaScript es más manejable. El ejemplo anterior de salida del formulario podríamos definirlo así: por un lado, definiríamos nuestra función JavaScript (en la propia página HTML o en un archivo aparte):

```
function mensajeSalida()
```



```
{  
    alert("Saliendo del formulario");  
}
```

Por otro lado, definiríamos así el atributo con el evento:

```
<form action="pagina.php" onmouseleave="mensajeSalida()">
```

En este caso, desde la función no podemos acceder al elemento `this` que provocó el evento, tendríamos que enviárselo como parámetro. Por ejemplo, así cambiaríamos el color de fondo de un `div` al entrar en él, usando funciones:

```
function cambiarColor(elemento)  
{  
    elemento.style.backgroundColor= "yellow";  
}  
  
<div onmouseenter="cambiarColor(this)">...</div>
```

5.3.3 Manejadores de eventos usando el DOM

En lugar de utilizar los atributos XHTML vistos antes, podemos acceder a los diferentes elementos del DOM y definir los eventos sobre ellos. Tiene la ventaja de que el código HTML queda intacto (no tenemos que alterarlo) y además podemos hacer referencia a la variable `this` para acceder al elemento que provocó el evento. Por ejemplo, de este modo definiríamos el evento de cambio de color sobre el `div` anterior. En primer lugar, debemos identificar el elemento de algún modo (típicamente con un `id`):

```
<div id="div1">...</div>
```

Después, accedemos al elemento desde JavaScript (por ejemplo, con `document.getElementById`), y añadimos el evento sobre el propio elemento:

```
let div = document.getElementById("div1");  
div.onmouseenter = function() {  
    this.style.backgroundColor = "yellow";  
}
```



5.3.4 Manejadores de eventos con event listeners

Como cuarta y última forma de definir manejadores de eventos (y la más recomendable de todas) podemos definir listeners sobre los elementos de la página. Es algo parecido a la forma anterior usando el DOM, pero con algunas ventajas, como por ejemplo la posibilidad de definir distintos manejadores para un mismo evento.

Para utilizar esta opción, debemos emplear la instrucción `addEventListener`, pasándole 2 parámetros entre paréntesis:

- El nombre del evento (sin prefijo "on")
- La función que se va a ejecutar

El mismo evento anterior de entrada de ratón sobre el div lo podríamos definir así:

```
let div = document.getElementById("div1");
div.addEventListener('mouseenter', function() {
    this.style.backgroundColor = "yellow";
});
```

Como decíamos, podemos definir varias funciones manejadoras sobre el mismo evento. En este ejemplo cambiamos el color de fondo desde una función, y la anchura desde otra (aunque podríamos hacer las dos cosas desde la misma función también):

```
let div = document.getElementById("div1");
div.addEventListener('mouseenter', function() {
    this.style.backgroundColor = "yellow";
});
div.addEventListener('mouseenter', function() {
    this.style.width="50%";
});
```

También podemos definir las funciones aparte y referenciarlas desde `addEventListener`:

```
let cambiarColorFondo = function() {
    this.style.backgroundColor = "yellow";
}

let div = document.getElementById("div1");
div.addEventListener('mouseenter', cambiarColorFondo);
```



Puedes probar [aquí](#) un ejemplo de uso de eventos con cada una de las cuatro estrategias comentadas.

Ejercicio 4.1:

Crea una página llamada **01_eventos_imagenes.html**. Coloca en ella tres imágenes centradas, con un tamaño de 100 x 100 píxeles (mediante CSS). Haz que, al pasar el ratón por encima de cualquiera de ellas, se cambie su tamaño a 200 x 200 y se le ponga un borde rojo de 3 píxeles de grosor. Haz que este efecto desaparezca al salir el ratón de la imagen.

Ejercicio 4.2:

Crea una página llamada **2_evento_form.html** con un formulario como este:

Registro de usuario

Nombre:

Email:

Enviar

Define eventos JavaScript sobre los dos cuadros de texto para que:

- Si salimos del cuadro de texto (evento blur) y no tiene valor (propiedad value está vacía), se le ponga el color de fondo rojo al cuadro de texto. Si tiene valor, dejaremos el color de fondo blanco.
- Cuando entramos al cuadro de texto (evento focus), se le ponga el color de fondo amarillo.

4. Uso avanzado de eventos

5.3.5 Propagación de eventos

Cuando interactuamos con una página web, se producen una serie de eventos sobre los elementos de dicha página, como hemos podido ver en esta sesión. Cada vez que hacemos clic en un elemento, o escribimos en un control de formulario, o simplemente pasamos el ratón sobre una zona de la página, se desencadena un evento al que podemos responder.



Sin embargo, hemos de tener en cuenta que el contenido de una página está jerarquizado: los elementos que la componen están unos dentro de otros. Por ejemplo, una imagen puede encontrarse dentro de un párrafo, que a su vez se encuentre en un div o contenedor, que a su vez forme parte de una rejilla con otros contenedores. Esta jerarquía permite que varios elementos involucrados puedan responder a un evento; por ejemplo, si tenemos una imagen dentro de un párrafo, y a su vez dentro de un div, podemos hacer que, al pasar el ratón por una imagen, todos ellos (div, párrafo y/o imagen) puedan responder a este evento. Esto se puede afrontar desde dos puntos de vista:

- Bubbling: se responde al evento desde el elemento más específico (la imagen, en el ejemplo anterior) al menos específico (el div)
- Capturing: se responde al evento desde el elemento menos específico (el div en el ejemplo anterior) al más específico (la imagen)

Ejemplo de bubbling

Veamos el ejemplo anterior más concretamente. Imaginemos la siguiente estructura HTML:

```
<div id="div1">
  <p id="p1">
    
  </p>
</div>
```

Definimos en JavaScript el siguiente gestor de eventos `eventoClic`, y lo aplicamos por igual a los tres elementos:

```
let eventoClic = function() {
  alert("Has pulsado " + this.id);
}

let div = document.getElementById("div1");
let p = document.getElementById("p1");
let img = document.getElementById("img1");

div.addEventListener("click", eventoClic);
p.addEventListener("click", eventoClic);
img.addEventListener("click", eventoClic);
```

Con esta estructura, ocurre lo siguiente:



- Si hacemos clic en el div (fuera del párrafo y la imagen), saca el mensaje Has pulsado div1
- Si hacemos clic en el párrafo (fuera de la imagen), saca los mensajes Has pulsado p1 y Has pulsado div1
- Si hacemos clic en la imagen, saca los tres mensajes: Has pulsado img1, Has pulsado p1 y Has pulsado div1

Este es el comportamiento por defecto de los eventos, que se corresponde con el bubbling: se propagan desde el más específico donde se ha producido hasta el más general que lo contiene.

Ejemplo de capturing

Si queremos ofrecer el comportamiento opuesto, es decir, que los eventos se propaguen desde el más general al más específico, debemos proporcionar un tercer parámetro true al event listener. Nuestro ejemplo anterior quedaría así:

```
div.addEventListener("click", eventoClic, true);
p.addEventListener("click", eventoClic, true);
img.addEventListener("click", eventoClic, true);
```

Ahora, si hacemos clic en la imagen los mensajes se muestran en orden inverso: desde el div hasta la imagen. Lo mismo ocurre si hacemos clic en el párrafo (primero se muestra el mensaje del div y luego el del párrafo).

Ejercicio 4.3

Crea una página llamada **03_bubbling_capturing.html** y prueba el código de los ejemplos anteriores con la imagen que quieras. Define un color de fondo y un padding para el div y para el párrafo, de forma que te ayude a localizarlos mejor en la página y ver donde tienes que hacer clic en cada caso.

5.3.6 Obtener información del evento

En algunas ocasiones nos puede ser útil obtener información del evento que se produce. Por ejemplo, las coordenadas donde hemos hecho clic con el ratón, o qué tecla hemos pulsado en un evento de teclado. Para recoger información del evento podemos pasarlo como parámetro a la función manejadora.

```
let evento = function(event)
{
    ...
}
```



En la variable donde hemos recogido el evento (event en el ejemplo anterior) tendremos disponibles una serie de propiedades según el tipo de evento que haya sido. Algunas de las más útiles son:

- Si ha sido un evento de teclado (pulsar una tecla, por ejemplo), en la propiedad `key` tendremos el nombre de la tecla pulsada, y en `keyCode` tendremos el código interno de la tecla en Unicode, aunque esta última propiedad puede no funcionar adecuadamente en ciertos navegadores con `keyup` o `keydown`. También podemos detectar con algunos booleanos especiales como `shiftKey`, `ctrlKey`, etc, si se ha pulsado alguna tecla especial (como Mayúsculas o Control, por ejemplo):

```
document.addEventListener("keydown", function(evento)
{
    if (evento.ctrlKey && evento.key == 'z')
    {
        alert("Has pulsado Ctrl + Z");
    }
    else if (evento.shiftKey)
    {
        alert("Mayúsculas pulsadas");
    }
});
```

- Si ha sido un evento de ratón, en las propiedades `clientX` y `clientY` tendremos las coordenadas del ratón con respecto a la ventana del navegador, y en `screenX` y `screenY` tendremos las coordenadas respecto a la pantalla. El siguiente ejemplo recoge en qué coordenadas se hace clic con el ratón, en un documento.

```
document.addEventListener("click", function(event)
{
    let mouseX = event.clientX;
    let mouseY = event.clientY;
    alert(mouseX + ", " + mouseY);
})
```

Ejercicio 4.4:

Crea una página llamada **04_info_eventos.html**. Define en ella un div con un borde negro, y que dentro de ese div se informe de las coordenadas actuales del ratón dentro de la ventana. Si pulsamos cualquier tecla, deberemos mostrar un alert con su código.



5.3.6.1 Otras propiedades de los eventos

Además de las propiedades particulares de cada tipo de evento (coordenadas del ratón, tecla pulsada, etc) existen una serie de propiedades y operaciones generales para cualquier evento. Aquí mostramos algunas de las más importantes:

- target**: hace referencia al elemento HTML sobre el que se ha producido el evento
- type**: tipo de evento producido. Se tiene el nombre del evento, tal y como los hemos visto hasta ahora: click, mouseenter, etc.
- preventDefault()**: esta operación es útil para eventos que desencadenan una acción por defecto. Por ejemplo, el evento submit cuando pulsamos el botón de un formulario, o el evento de cargar un enlace al hacer clic sobre él. Al llamar a esta instrucción evitamos que se ejecute esa acción predefinida. Será útil, como veremos más adelante, para prevenir que se envíen formularios incorrectos, entre otras cosas.
- stopPropagation()**: evita que el evento se siga propagando a niveles superiores o inferiores (según si estamos en modo bubbling o capturing, respectivamente).

El siguiente manejador muestra el contenido HTML del elemento sobre el que se ha producido el evento, y el tipo de evento que se ha producido (click en este caso):

```
let informacion = function(event)
{
    alert(event.target.innerHTML);
    alert(event.type);
}
```

5.3.7 El evento de “arrastrar y soltar” (drag & drop)

El hecho de arrastrar elementos de una página de una zona a otra para ubicarlos en ciertos contenedores es algo habitual. Hasta hace no mucho, esto requería del uso de ciertas librerías externas como jQuery para simplificar la tarea, pero se ha convertido en algo tan habitual que la propia API de JavaScript ha incorporado esta característica al lenguaje, junto con HTML5.

El proceso se basa en tres eventos que debemos gestionar:

- dragstart**: cuando comenzamos a arrastrar un componente. Típicamente lo vamos a utilizar para guardarnos la referencia del elemento que estamos arrastrando, para recuperarla después al soltar. Esta referencia suele ser el id del elemento.



- dragover**: cuando estamos arrastrando el elemento por encima del componente donde lo podemos soltar. En este caso debemos evitar que se active la acción por defecto, que es rechazar el elemento, usando `evento.preventDefault()`.
- drop**: cuando finalmente se suelta el elemento. Si lo soltamos sobre un componente que no hemos configurado, el propio componente rechazará al elemento, que volverá a su lugar de origen. Pero si hemos configurado adecuadamente el evento **dragover** sobre ese elemento, podemos simplemente usar este otro evento para que ese componente “acoe” al elemento que hemos soltado, añadiéndolo como hijo (`appendChild`).
- Todo esto va unido a un atributo HTML llamado `draggable`, que debemos poner a `true` en los elementos que queramos poder arrastrar.

Aquí vemos un sencillo ejemplo donde podemos arrastrar el logo de JavaScript a una caja. El HTML puede quedar así:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo Drag & Drop</title>
  <style>
    #contenedor
    {
      display: grid;
      grid-template-columns: 1fr 1fr;
      gap: 20px;
    }
    #receptor
    {
      border: 1px solid black;
      border-radius: 10px;
      height: 300px;
      text-align: center;
      vertical-align: middle;
    }
  </style>
</head>
<body>
  <div id="contenedor">
    <div>
      
    </div>
    <div id="receptor">
  </div>
```



```
</div>
</body>
</html>
```

Y el código JavaScript asociado sería este:

```
let imagen = document.getElementById("logo");
let contenedor = document.getElementById("receptor");

imagen.addEventListener("dragstart", function(evento) {
    evento.dataTransfer.setData("elemento", evento.target.id);
});
contenedor.addEventListener("dragover", function(evento) {
    evento.preventDefault();
});
contenedor.addEventListener("drop", function(evento) {
    evento.preventDefault();
    let datos = evento.dataTransfer.getData("elemento");
    evento.target.appendChild(document.getElementById(datos));
});
```

Explicamos el código que hemos definido antes:

- Definimos el atributo `draggable` sobre la imagen para poderla arrastrar
- Definimos el evento `dragstart` sobre la imagen, y guardamos en el evento el id de la imagen, en su propiedad `dataTransfer`, asignándole el nombre `elemento`, por ejemplo. Dicha propiedad se utiliza para almacenar y recuperar información utilizada durante el proceso de arrastrar y soltar.
- Definimos el evento `dragover` sobre la caja para que, cuando algo esté "sobrevolándola" no lo rechace (`preventDefault`)
- Definimos el evento `drop` también sobre la caja para anular el evento por defecto (`preventDefault`), acceder al elemento que hemos estado arrastrando (recuperamos la información del `dataTransfer`) y añadirlo (`appendChild`) al contenido de la caja. Puede resultar extraño usar aquí `preventDefault` también, pero es que en algunos navegadores la acción por defecto cuando se arrastra un componente en alguna parte de la página es mostrar ese componente (en este caso, mostrar la imagen) a pantalla completa.

Ejercicio 5.5

Crema una página llamada **05_drag_drop_js.html** y utiliza [esta plantilla](#) como base. Se pide que añadas el código JavaScript necesario para poder arrastrar los logos de los navegadores cada uno a su casilla correspondiente. En principio puedes arrastrar cualquier logo a cualquier casilla, pero al pulsar el botón de Comprobar se debe mostrar un alert indicando si has acertado o no.



5.3.8 Encapsulando el código JavaScript

El uso de JavaScript en páginas HTML es una herramienta muy potente, que nos permite enriquecer el comportamiento de las páginas añadiendo elementos dinámicamente: modificar contenidos, responder a eventos de usuario, etc. Sin embargo, en ocasiones puede ser conveniente proteger nuestro código JavaScript, de forma que quede aislado. Esto puede resultar particularmente útil cuando accedemos a distintos archivos JavaScript, para así evitar colisiones en nombres de variables o funciones. Para ello, tenemos dos alternativas.

Evento load

Como primera alternativa, podemos hacer que nuestro código se cargue dentro del evento load de la página. Esto hará, por un lado, que ciertas instrucciones no se ejecuten hasta que los contenidos estén cargados (útil si queremos acceder a partes del DOM), y por otro lado, que todo el código quede encapsulado en el evento en sí, y no sea accesible desde fuera.

El ejemplo anterior de bubbling y capturing lo podríamos definir así dentro del evento load de la ventana (y por tanto, este código lo podríamos poner tanto en la cabecera como en el cuerpo de la página):

```
window.addEventListener('load', function()
{
    let eventoClic = function() {
        alert("Has pulsado " + this.id);
    }

    let div = document.getElementById("div1");
    let p = document.getElementById("p1");
    let img = document.getElementById("img1");

    div.addEventListener("click", eventoClic);
    p.addEventListener("click", eventoClic);
    img.addEventListener("click", eventoClic);
});
```

IIFE

Como alternativa a la opción anterior, también podemos emplear una estructura normalmente conocida como **IIFE** (Immediately Invoked Function Expression). Básicamente consiste en encapsular todo nuestro código en una función que se auto-ejecuta. De este modo, el código interno a la función no es visible desde fuera, y evitamos así que colisione con otros fragmentos de código JavaScript que podamos querer incorporar.



Así quedaría el ejemplo anterior de bubbling y capturing encapsulado con IIFE:

```
(function()
{
    let eventoClic = function() {
        alert("Has pulsado " + this.id);
    }

    let div = document.getElementById("div1");
    let p = document.getElementById("p1");
    let img = document.getElementById("img1");

    div.addEventListener("click", eventoClic);
    p.addEventListener("click", eventoClic);
    img.addEventListener("click", eventoClic);
})();
```

Observemos que hemos encapsulado el código en una función que se auto-ejecuta. Esto hace que los elementos que hemos definido (función eventoClic, variables div, p, img...) no puedan ser accesibles desde fuera de este código. Además, este código se ejecuta inmediatamente, con lo que, en este caso, convendría colocarlo después de cargar la página (para poder acceder a los elementos del DOM).

Ejercicio 5.6

Crea una versión del ejercicio 04 en otro archivo llamado **06_info_eventos_iife.html** y encapsula el código JavaScript utilizando IIFE.

Ejercicio 5.1 (antiguo material)

A partir de la página web proporcionada, completar el código JavaScript para que:

1. Cuando se pinche sobre el primer enlace, se oculte su sección relacionada
2. Cuando se vuelva a pinchar sobre el mismo enlace, se muestre otra vez esa sección de contenidos
3. Completar el resto de enlaces de la página para que su comportamiento sea idéntico al del primer enlace
4. Cuando una sección se oculte, debe cambiar el mensaje del enlace asociado (pista: propiedad `innerHTML`)

Ejercicio 5.2 (antiguo material)

Completar el código JavaScript proporcionado para que:



1. Al mover el ratón en cualquier punto de la ventana del navegador, se muestre la posición del puntero respecto del navegador y respecto de la página:



Figura 6.1. Información que se muestra para los eventos del ratón

Para mostrar los mensajes, utilizar la función `muestraInformacion()` deduciendo su funcionamiento a partir de su código fuente.

2. Al pulsar cualquier tecla, el mensaje mostrado debe cambiar para indicar el nuevo evento y su información asociada:



Figura 6.2. Información que se muestra para los eventos del teclado

3. Añadir la siguiente característica al script: cuando se pulsa un botón del ratón, el color de fondo del cuadro de mensaje debe ser amarillo (#FFFFCC) y cuando se pulsa una tecla, el color de fondo debe ser azul (#CCE6FF). Al volver a mover el ratón, el color de fondo vuelve a ser blanco.



Figura 6.3. El color de fondo del cuadro de información cambia en función del tipo de evento

Ejercicio 5.3 (antiguo material)

Crear un script que informe al usuario en que zona de la pantalla ha pulsado el ratón. Las zonas definidas son las siguientes: izquierda arriba, izquierda abajo, derecha arriba y derecha abajo. Para determinar el tamaño de la ventana del navegador, utilizar la función `informacion()` (que está sin terminar), proporcionada.



6 Tema 6 - Validación de formularios

Otra de las principales funcionalidades de JavaScript es poder validar el contenido de un formulario antes de enviarlo al servidor, ahorrándole a éste una buena parte del trabajo de procesamiento de datos. De hecho, esta fue una de las razones por las que se inventó JavaScript, aunque hoy en día esta tarea también puede hacerse total o parcialmente (según el formulario) utilizando HTML5. Veremos en este documento cómo podemos combinar ambas tecnologías (JavaScript y HTML5) para la validación de formularios en el lado del cliente.

6.1 Validación tradicional de formularios

Antes de la aparición de HTML5, JavaScript era la única herramienta disponible en el lado del cliente para poder validar formularios. Veremos en este apartado algunos elementos de JavaScript que podemos tener en cuenta a la hora de realizar esta validación, y en el siguiente apartado veremos cómo podemos combinarlos con la validación nativa de HTML5.

6.1.1 Acceso a los datos del formulario

Ya hemos visto que para poder acceder al contenido de un documento HTML hemos de utilizar las funciones de acceso al DOM. Podemos asignar un id diferente a cada elemento del formulario y acceder a él con `document.getElementById(...)`, pero también podemos emplear el atributo `name` de cada campo del formulario (y del propio formulario) para acceder a ellos más directamente. Por ejemplo, para acceder al siguiente formulario:

```
<form name="miFormulario" action="pagina.php" method="post">
...
</form>
```

Usaríamos el código `document.miFormulario` (una vez el formulario ya esté cargado). Si dentro del formulario tenemos, por ejemplo, un campo de texto llamado email:

```
<form name="miFormulario" action="pagina.php" method="post">
...
    <input type="email" id="email" name="email" size="20">
...
</form>
```



accederíamos a él con el código `document.miFormulario.email`. Es decir, usamos el atributo `name` tanto del formulario al que queremos acceder como del campo concreto que queremos consultar. También podríamos utilizar el método `getElementById` visto antes para acceder a elementos del contenido, siempre que les pongamos un `id`. En el caso del campo `email` anterior, accederíamos directamente con `document.getElementById("email")`.

6.1.2 Consulta y comprobación de valores

Una vez hemos accedido al formulario o a alguno de sus campos (normalmente se accede a los campos), todos ellos tienen una serie de propiedades básicas cuyos valores podemos consultar o modificar (según el caso):

- `type`: indica el tipo de elemento que se trata. Para los elementos de tipo `input`, coincidirá con su atributo `type`. Para listas, será `select-one` (selección simple) o `select-multiple` (selección múltiple), y para las áreas de texto será `textarea`.
- `name`: indica el nombre del elemento (su atributo `name`)
- `value`: indica el valor que tiene actualmente el elemento. Esta propiedad puede tanto leerse como modificarse, por lo que desde JavaScript podemos dar valor a los campos de un formulario.

El siguiente ejemplo comprueba si el campo `email` anterior está vacío, y si es así, le asigna automáticamente un e-mail por defecto `admin@empresa.com`:

```
let mail = document.miFormulario.email;
if (mail.value == null || mail.value == "")
{
    mail.value="admin@empresa.com";
}
```

El caso de los botones de radio y los checkbox

En el caso de estos dos campos, además de las propiedades anteriores tenemos una propiedad más llamada `checked`, que indica si el botón o el cuadro está marcado o no. El siguiente ejemplo comprueba si una casilla de checkbox llamada `casado` está marcada, mostrando un mensaje en caso afirmativo:

```
let casado = document.miFormulario.casado;
if (casado.checked)
{
    alert("Has indicado que estás casado");
}
```



El caso concreto de los botones de radio es algo más complejo, ya que todos comparten el mismo name. Veamos este formulario:

```
<form name="miFormulario" action="pagina.php" method="post">
  <input type="radio" name="idioma" value="ESP" />Español<br />
  <input type="radio" name="idioma" value="ENG" />Inglés<br />
  <input type="radio" name="idioma" value="FRA" />Francés<br />
  ...
</form>
```

Si quisiéramos acceder a `document.miFormulario.idioma`, accederíamos a los tres botones de radio a la vez. Tendríamos que utilizar una estructura for o similar para recorrer los botones y ver cuál está marcado. Aquí tenemos un ejemplo de eso:

```
let listaBotones = document.miFormulario.idioma;
for (let indice in listaBotones)
{
  if (listaBotones[indice].checked)
  {
    alert ("Has seleccionado " + listaBotones[indice].value);
  }
}
```

La propiedad value de cada botón será igual al atributo value de dicho botón (ESP, ENG o FRA, respectivamente, para el ejemplo anterior). También podemos, de forma abreviada, acceder al value del control, y obtendremos el valor actualmente seleccionado:

```
let idiomaSeleccionado = document.miFormulario.idioma.value;
```

El caso de las listas

En el caso de una lista (select), tenemos las propiedades:

- `selectedIndex`, que nos da la posición del elemento actualmente seleccionado
- `options`, que es una lista con todas las opciones, que podemos recorrer con una estructura for o similar
-

El siguiente ejemplo obtiene el elemento seleccionado de una lista con id lista1, y nos muestra con un mensaje su valor y el texto que hay visible en la lista (recordemos que cada option de la lista puede tener un value, y dentro de la etiqueta, su propio texto diferente del value):



```
let lista = document.getElementById("lista1");
let indiceSeleccionado = lista.selectedIndex;
let elementoSeleccionado = lista.options[indiceSeleccionado];
alert ("Has seleccionado " + elementoSeleccionado.text);
alert ("Su valor es " + elementoSeleccionado.value);
```

Nuevamente, también podemos acceder al value de la lista y obtendremos el valor actualmente seleccionado:

```
let valorSeleccionado = lista.value;
```

Ejercicio 1:

Crea un formulario en una página llamada **form1_js.html**. Este formulario tendrá por nombre form1, y dentro tendrá los siguientes campos:

- Un campo de texto llamado login, con una etiqueta asociada que ponga "Login".
- Un campo numérico llamado nhijos, con una etiqueta asociada que ponga "Número de hijos"
- Una casilla checkbox llamada info con un texto que ponga "Deseo recibir información por e-mail"
- Tres botones de radio que llamados horario con los textos Mañana, Tarde y Noche respectivamente, y encima de ellos una etiqueta asociada que ponga "Horario preferido para llamadas"
- Una lista desplegable llamada operador, y dentro los valores Movil1, Taronje, Hablafone y Roigo. Encima debe tener una etiqueta que diga "Seleccione operador"
- Un botón (tipo button) de Enviar que, al hacer clic sobre él, active una función llamada mostrarInformacion que saque con mensajes (alert) cada una de las opciones que hayamos escrito en cada campo del formulario. Por ejemplo, si hemos puesto como login "pepe", como número de hijos 2, hemos marcado la casilla de información, elegido turno de Tarde y el operador Taronje, deberá sacar, uno a uno, los siguientes mensajes:
 - Tu login es pepe
 - Tienes 2 hijos
 - Deseas recibir información
 - Has elegido el horario de Tarde
 - Has elegido el operador Taronje
- En el caso de que el número de hijos esté vacío, deberá rellenarlo con el valor 0 al pulsar el botón de Enviar



6.1.3 Algunas opciones de validación de campos

Vamos a ver algunas de las opciones más habituales para validar campos de formularios, aunque la mayoría de ellas ya las podemos hacer directamente utilizando HTML5.

Comprobación de campo obligatorio vacío

Para comprobar que un campo obligatorio está vacío, tenemos el atributo `required` en HTML5.

Para validarlo desde JavaScript, debemos acceder a dicho campo (por su id o su nombre, como queramos), y comprobar si su valor es igual a `""`:

```
let login = document.miFormulario.login;
if (login.value == "")
{
    alert ("El campo login no puede estar vacío");
}
```

Comprobación de tipo de dato adecuado

Para comprobar que en un campo hemos introducido un tipo de dato adecuado, en muchas ocasiones podemos emplear el atributo `type` específico en la etiqueta `input`. Por ejemplo, con `type="number"` nos aseguramos de que el campo tendrá un valor numérico, y también podemos establecer el rango de valores permitido con los atributos `min` y `max`. Pero, además de lo anterior, existen algunas funciones auxiliares en JavaScript para hacer estas comprobaciones. Lo más habitual es comprobar que un campo numérico (por ejemplo, una edad, o un año), tenga realmente un número y no otra cosa. Para ello usaremos la función `isNaN` para comprobar si ese dato no es numérico. Así, por ejemplo, si tenemos un campo `edad` y queremos ver si el dato introducido en él es numérico, tendríamos algo como:

```
let edad = document.miFormulario.edad;
if (isNaN(edad.value))
{
    alert ("El dato introducido en el campo edad no es un número");
}
```

Comprobación de selección en lista

Ya hemos visto que con la propiedad `selectedIndex` obtenemos la posición del elemento seleccionado en una lista. Si el usuario no ha seleccionado ninguna opción, o tiene seleccionada la primera de todas (que normalmente es un valor por defecto que hay que cambiar), esta propiedad vale 0 o `null` (según el tipo de lista), así que basta comprobar estos dos valores para ver si se ha elegido algo de la lista:



```
let lista = document.miFormulario.lista;
if (lista.selectedIndex == null || lista.selectedIndex == 0)
{
    alert ("Debes seleccionar algún elemento de la lista");
}
```

Comprobación de patrón. Expresiones regulares

Para comprobar que un determinado campo cumple un determinado patrón (por ejemplo, que tenga cuatro dígitos y una letra), se utilizan expresiones regulares, similares a las que existen para HTML5 (mediante el atributo pattern). En la siguiente tabla tenemos un resumen de los tipos de símbolos que podemos utilizar en las expresiones regulares:

Símbolo	Significado
^	Se utiliza para indicar el comienzo de un texto o línea
\$	Se utiliza para indicar el final de un texto o línea
*	Indica que el carácter anterior puede aparecer 0 o más veces
+	Indica que el carácter anterior puede aparecer 1 o más veces
?	Indica que el carácter anterior puede aparecer 0 o 1 vez
.	Representa a cualquier carácter, salvo el salto de línea
x y	Indica que puede haber un elemento x o y
{n}	Indica que el carácter anterior debe aparecer n veces
{m, n}	Indica que el carácter anterior debe aparecer entre m y n veces
[abc]	Cualquiera de los caracteres entre corchetes. Podemos especificar rangos con un guión, como por ejemplo [A-L]
[^abc]	Cualquier carácter que no esté entre los corchetes
\b	Un límite de palabra (espacio, salto de línea...)
\B	Cualquier cosa que no sea un límite de palabra
\d	Un dígito (equivaldría al intervalo [0-9])
\D	Cualquier cosa que no sea un dígito (equivaldría a [^0-9])
\n	Salto de línea



Símbolo	Significado
\s	Cualquier carácter separador (espacio, tabulación, salto de página o de línea)
\S	Cualquier carácter que no sea separador
\t	Tabulación
\w	Cualquier alfanumérico incluyendo subrayado (igual a [A-Za-z0-9_])
\W	Cualquier no alfanumérico ni subrayado ([^A-Za-z0-9_])

Algunos símbolos especiales (como el punto, o el +), se representan literalmente anteponiéndoles la barra invertida. Por ejemplo, si queremos indicar el carácter del punto, se pondría \. Veamos algunos ejemplos:

Ejemplo	Significado
^\d{9,11}\$	Entre 9 y 11 dígitos
(?!^[0-9]*\$)(?!^[a-zA-Z]*\$)^[a-zA-Z0-9]{8,10}\$	Password seguro (8-10 caracteres, con al menos un dígito y una letra)
^\d{1,2}\/\d{1,2}\/\d{2,4}\$	Fecha (d/m/a)
^[\w-\._]{3,}@([\w-]{2,}\.)*([\w-]{2,}\.)*[\w-]{2,4}\$	E-mail

El siguiente ejemplo comprueba si en un campo llamado dni tenemos un DNI válido, formado por 8 dígitos y una letra mayúscula:

```
let dni = document.miFormulario.dni.value;
if (/^\d{8}[A-Z]$/.test(dni) == false)
{
    alert ("El DNI es incorrecto");
}
```

Observa que ponemos primero la expresión (entre dos barras /), seguida del método test y la variable que queremos comprobar si encaja en ese patrón.

6.1.4 Cómo validar todo el formulario

¿Cómo aunar todo lo que hemos visto y validar las opciones del formulario que queramos antes de enviarlo? Primero debemos tener en cuenta que el evento que se dispara cuando intentamos



enviar el formulario es el evento `submit`. Por tanto, debemos añadir un event handler sobre el formulario, llamando a la función que queramos usar para validar. En la función podemos hacer uso del método `preventDefault`, por ejemplo, para evitar que el formulario se envíe si hay algún error.

```
document.miFormulario.addEventListener("submit", validar);

...
function validar(evento)
{
    let formularioCorrecto = true;

    // Código para comprobar cada campo
    // Pondríamos "formularioCorrecto" a false si alguno falla

    if (!formularioCorrecto)
        evento.preventDefault();
}
```

Ejercicio 2:

Crea un formulario en un documento llamado **form2_js.html**. El formulario deberá llamarse `form2`, y deberás definir una función llamada `validarFormulario` que comentaremos a continuación. El formulario debe tener los siguientes campos:

- Un campo llamado `login`, con una etiqueta asociada "Login"
- Un campo llamado `edad` (numérico), con una etiqueta asociada "Edad"
- Un campo llamado `nacimiento` (numérico) con una etiqueta "Año de nacimiento"
- Una lista desplegable llamada `provincia` con una etiqueta "Seleccione provincia". La primera opción de esa lista debe decir "-Seleccionar una provincia-", y tener `value=""`. Las otras tres opciones deben ser Alicante, Valencia y Castellón.
- Un botón de tipo `submit` y con valor Enviar.

Al intentar enviar el formulario, se activará la función `validarFormulario()` comentada antes, que deberemos implementar para comprobar lo siguiente:

- El campo `login` no puede estar vacío
- El campo `edad` puede estar vacío, pero si no lo está, debe tener un número entre 0 y 200



- El campo nacimiento debe tener 4 dígitos
- La lista desplegable debe tener un elemento seleccionado que no sea el primero.
- Si todo lo anterior se cumple, se deberá mostrar un mensaje "Todo es correcto" y enviar el formulario (el envío fallará porque la página destino no la habremos implementado). Si algún campo no es válido, se deberá mostrar el correspondiente mensaje de error y evitar el envío.

2. Validación de formularios combinada con HTML5

Con la aparición de HTML5, se dotó a este lenguaje de marcado de la posibilidad de validar la información introducida en los campos de los formularios, a través de atributos como:

- required**: determina que un campo es obligatorio
- pattern**: obliga a que los datos de un campo tengan un patrón determinado
- min**, **max**: determina los límites inferior superior en campos con valores numéricos
- Además, algunos tipos de campos (e-mail, URL...) tienen una auto-validación para comprobar que la información que hemos introducido se puede corresponder con lo que ese campo espera (un e-mail, una dirección web, etc, según el caso).

Podéis consultar [aquí](#) más información sobre cómo aplicar estos atributos en los campos de un formulario. En cuanto al atributo pattern, la sintaxis que podemos seguir para definir las expresiones regulares es la misma que la que hemos explicado en este documento unos párrafos atrás.

Sin embargo, la validación que ofrece HTML5 se queda un poco corta en dos aspectos fundamentalmente:

- Tiene sus limitaciones a la hora de validar cierta información algo más específica o compleja. Por ejemplo, comprobar que dos campos tengan el mismo valor, como sería el caso de una confirmación de password en un formulario.
- Ofrece unos mensajes de error predefinidos ante los errores de validación (no configurables)

Afortunadamente, podemos incorporar nuestro código JavaScript en nuestras webs y combinarlo con la validación de HTML5 para suplir esas carencias, de forma que todo quede integrado en un único mecanismo de validación que aúna las ventajas de ambos (HTML5 y JavaScript).



6.1.5 Validaciones personalizadas

Como comentábamos antes, una de las deficiencias más importantes que ofrece la validación de formularios con HTML5 es que es algo limitada. Si, por ejemplo, queremos comprobar que el usuario ha puesto el mismo password en dos campos distintos para asegurarnos de que no se ha equivocado al escribirlo, no disponemos de ningún atributo específico para esto. Sin embargo, gracias a JavaScript, y en concreto al método `setCustomValidity` podemos controlar casos como éste, e incluso personalizar el mensaje que se mostrará al usuario si se produce un error de validación.

Este método recibe como parámetro el mensaje de error que mostraremos en el campo afectado. De este modo, cuando intentemos enviar el formulario, el envío se detendrá si alguno de los campos tiene algún mensaje de error pendiente. Para anular el mensaje y volver a decir que todo está correcto, llamaremos de nuevo al método pasándole una cadena vacía.

El siguiente ejemplo comprueba si dos campos de password `pass1` y `pass2` son iguales. Si no lo son, establece el mensaje "Los passwords no coinciden" en el segundo campo. Si todo es correcto, vuelve a dejar el mensaje vacío.

Así sería el formulario del ejemplo:

```
<form action="pagina.php" name="form1" id="form1">
  <label>
    Login:
    <input type="text" name="login" id="login" required>
  </label><br>
  <label>
    Password:
    <input type="password" name="pass1" id="pass1" required>
  </label><br>
  <label>
    Repetir password:
    <input type="password" name="pass2" id="pass2" required>
  </label><br>
  <input type="submit" value="Enviar" id="enviar">
</form>
```

Y así definiríamos el código JavaScript para gestionar la validación de los passwords:

```
let pass1 = document.getElementById("pass1");
let pass2 = document.getElementById("pass2");
```



```
pass1.addEventListener("input", validarPass);
pass2.addEventListener("input", validarPass);

function validarPass()
{
    if (pass1.value != pass2.value)
        pass2.setCustomValidity("Los passwords no coinciden");
    else
        pass2.setCustomValidity("");
}
```

Notar que, si cargamos el formulario y dejamos vacío el login o los passwords, se sigue disparando el error de que es obligatorio. Dicho de otro modo, la función de validación nos ayuda a completar las validaciones que hayamos indicado con HTML5, sin que éstas se vean anuladas.

6.1.6 Personalizar mensajes de error

A la hora de modificar el mensaje de error por defecto de un campo que se valida automáticamente, también podemos hacer uso del método `setCustomValidity`. En este caso, podemos hacer uso del evento `invalid` que se genera cuando el campo no es válido, y tendremos que asegurarnos de vaciar el mensaje de error después (por ejemplo, en cuanto se vuelva a escribir algo en el campo):

```
let login = document.getElementById("login");
login.addEventListener("invalid", function() {
    this.setCustomValidity("El login es obligatorio");
});
login.addEventListener("input", function() {
    this.setCustomValidity("");
});
```

En algunos casos nos puede interesar mostrar distintos mensajes de error dependiendo del tipo de fallo. Por ejemplo un campo e-mail obligatorio puede fallar porque esté vacío, o porque el valor que hayamos introducido no se corresponda con un e-mail válido. Los campos del formulario disponen, a través de la propiedad `validity`, de una serie de subpropiedades que podemos consultar para comprobar qué ha pasado exactamente. Algunas de estas subpropiedades son:

- `valid` es true cuando el campo actual es válido
- `valueMissing` es true cuando es un campo obligatorio vacío



- `typeMismatch` es true cuando el tipo de información no se corresponde con lo que espera el campo (por ejemplo, e-mails no válidos)
- `patternMismatch` es true cuando la entrada no se corresponde con lo especificado en un atributo `pattern`
- ...

El siguiente ejemplo muestra un mensaje u otro en un campo e-mail con id `email`, dependiendo del error producido:

```
let email = document.getElementById("email");
email.addEventListener("invalid", function() {
    if (email.validity.valueMissing)
        email.setCustomValidity("El e-mail no puede estar vacío");
    else if (email.validity.typeMismatch)
        email.setCustomValidity("El formato de email no es correcto");
});
email.addEventListener("input", function() {
    email.setCustomValidity("");
});
```

6.1.7 CSS y validación de formularios

Existen algunos selectores (más concretamente, pseudo-clases) en CSS que podemos emplear para definir ciertos estilos relacionados con la validación de formularios.

- La pseudo-clase `:required` se aplicará a los campos de formularios que sean requeridos. Por ejemplo, de este modo podemos ponerlos con un fondo amarillo:

```
input:required
{
    background-color: yellow;
}
```

- Las pseudo-clases `:valid` e `:invalid` se aplican sobre campos cuyo valor sea válido o inválido, respectivamente.

```
input:valid
{
    background-color: green;
}
```



```
input:invalid
{
    background-color: red;
}
```

Notar que en ocasiones estas pseudo-clases pueden solaparse. Si un campo es obligatorio e inválido e intentamos cambiar la misma propiedad desde las dos reglas (por ejemplo el color de fondo, como en el caso anterior), sólo prevalecerá una de ellas (la que se aplique más tarde). En este sentido, conviene que los estilos de campo obligatorio no se solapen con los de validez, y afecten a otras propiedades (color del borde, por ejemplo).

Ejercicio 3:

Crea un formulario en un documento llamado **form3_js.html**. El formulario deberá llamarse form3, y contendrá estos campos con sus respectivas restricciones:

- Un campo llamado login, obligatorio
- Dos campos de tipo password para introducir el password. Serán obligatorios, y además los dos passwords deben coincidir, y estar formados por al menos 8 caracteres, compuestos por letras (minúsculas o mayúsculas) y dígitos.
- Un campo llamado email, obligatorio
- Un botón de tipo submit y con valor Enviar.

Se debe comprobar la validez de los campos, y mostrar mensajes de error en formato HTML5 (sin alerts). Deberás configurar mensajes de error personalizados en el caso del password (para mostrar si no coinciden, o para indicar el patrón que debe tener). También se deben marcar con fondo rojo los controles que hayan provocado error al intentar enviar el formulario. Dicho color de fondo rojo se restablecerá a blanco en cuanto se vuelva a escribir algo sobre el control.